

22 Security and Incident Response Plan

Documentación normativa de Cartridge & Cloud

Proyecto

Cartridge & Cloud

Versión del proyecto

0.0.26

Autor

Blas Luis Rocha González / VRM Games

Versión documental

2.0

Estado

Baseline aprobada y consolidada

Última revisión

17 de julio de 2026

Índice

22 Security and Incident Response Plan	19
27 — Security and Incident Response Plan	19
Cómo leer los estados	19
Resumen de postura	20
0. Propósito, alcance y autoridad	20
Propósito	20
Autoridad	20
Alcance	20
Objetivos de seguridad	20
Estado del documento	20
1. Genealogía histórica de seguridad	21
Regla de consolidación	21
2. Estado de seguridad actual	21
Resumen ejecutivo	21
Interpretación	22
Prioridades inmediatas	22
3. Principios Secure by Design	22
4. Gobernanza y responsabilidad	22
Owner y modelo de operación	22
Responsabilidades mínimas	23
Separación de funciones proporcional	23
Revisión	23
5. Modelo de amenazas	23
Actores considerados	23
Objetivos del adversario o del fallo	23
Supuestos	23
Método	23
6. Activos que proteger	23
7. Superficies de ataque	24
8. Límites de confianza	24
9. Clasificación de información	24
10. Código fuente	25
Objetivo y decisión	25
Estado actual comprobable	25
Controles y procedimiento	25
Evidencia y criterio de gate	25
Riesgos y tratamiento	25
Trazabilidad	26
11. Repositorios y ramas	26
Objetivo y decisión	26
Estado actual comprobable	26
Controles y procedimiento	26
Evidencia y criterio de gate	26
Riesgos y tratamiento	26
Trazabilidad	26
12. Credenciales y secretos	26
Objetivo y decisión	27
Estado actual comprobable	27
Controles y procedimiento	27
Evidencia y criterio de gate	27
Riesgos y tratamiento	27
Trazabilidad	27
13. Steamworks y cuentas de partner	27
Objetivo y decisión	27
Estado actual comprobable	28

Controles y procedimiento	28
Evidencia y criterio de gate	28
Riesgos y tratamiento	28
Trazabilidad	28
14. Unity ID y servicios	28
Objetivo y decisión	28
Estado actual comprobable	28
Controles y procedimiento	28
Evidencia y criterio de gate	29
Riesgos y tratamiento	29
Trazabilidad	29
15. GitHub y dispositivos	29
Objetivo y decisión	29
Estado actual comprobable	29
Controles y procedimiento	29
Evidencia y criterio de gate	30
Riesgos y tratamiento	30
Trazabilidad	30
16. Estaciones de trabajo	30
Objetivo y decisión	30
Estado actual comprobable	30
Controles y procedimiento	30
Evidencia y criterio de gate	30
Riesgos y tratamiento	31
Trazabilidad	31
17. Blender y herramientas DCC	31
Objetivo y decisión	31
Estado actual comprobable	31
Controles y procedimiento	31
Evidencia y criterio de gate	31
Riesgos y tratamiento	31
Trazabilidad	32
18. Paquetes y dependencias	32
Objetivo y decisión	32
Estado actual comprobable	32
Controles y procedimiento	32
Evidencia y criterio de gate	32
Riesgos y tratamiento	32
Trazabilidad	32
19. Supply chain de Unity Package Manager	33
Objetivo y decisión	33
Estado actual comprobable	33
Controles y procedimiento	33
Evidencia y criterio de gate	33
Riesgos y tratamiento	33
Trazabilidad	33
20. SBOM	33
Objetivo y decisión	33
Estado actual comprobable	34
Controles y procedimiento	34
Evidencia y criterio de gate	34
Riesgos y tratamiento	34
Trazabilidad	34
21. Licencias y seguridad	34
Objetivo y decisión	34
Estado actual comprobable	34
Controles y procedimiento	35
Evidencia y criterio de gate	35
Riesgos y tratamiento	35

Trazabilidad	35
22. Artefactos de build	35
Objetivo y decisión	35
Estado actual comprobable	35
Controles y procedimiento	35
Evidencia y criterio de gate	36
Riesgos y tratamiento	36
Trazabilidad	36
23. Integridad y checksums	36
Objetivo y decisión	36
Estado actual comprobable	36
Controles y procedimiento	36
Evidencia y criterio de gate	36
Riesgos y tratamiento	37
Trazabilidad	37
24. Firmas y procedencia	37
Objetivo y decisión	37
Estado actual comprobable	37
Controles y procedimiento	37
Evidencia y criterio de gate	37
Riesgos y tratamiento	37
Trazabilidad	38
25. Backups del repositorio	38
Objetivo y decisión	38
Estado actual comprobable	38
Controles y procedimiento	38
Evidencia y criterio de gate	38
Riesgos y tratamiento	38
Trazabilidad	38
26. Backups documentales	38
Objetivo y decisión	39
Estado actual comprobable	39
Controles y procedimiento	39
Evidencia y criterio de gate	39
Riesgos y tratamiento	39
Trazabilidad	39
27. Backups de herramientas y fuentes	39
Objetivo y decisión	39
Estado actual comprobable	40
Controles y procedimiento	40
Evidencia y criterio de gate	40
Riesgos y tratamiento	40
Trazabilidad	40
28. Recuperación ante pérdida total	40
Objetivo y decisión	40
Estado actual comprobable	40
Controles y procedimiento	41
Evidencia y criterio de gate	41
Riesgos y tratamiento	41
Trazabilidad	41
29. Control de acceso	41
Objetivo y decisión	41
Estado actual comprobable	41
Controles y procedimiento	42
Evidencia y criterio de gate	42
Riesgos y tratamiento	42
Trazabilidad	42
30. Principio de mínimo privilegio	42
Objetivo y decisión	42

Estado actual comprobable	42
Controles y procedimiento	42
Evidencia y criterio de gate	43
Riesgos y tratamiento	43
Trazabilidad	43
31. MFA	43
Objetivo y decisión	43
Estado actual comprobable	43
Controles y procedimiento	43
Evidencia y criterio de gate	43
Riesgos y tratamiento	44
Trazabilidad	44
32. Altas, bajas y cambios de acceso	44
Objetivo y decisión	44
Estado actual comprobable	44
Controles y procedimiento	44
Evidencia y criterio de gate	44
Riesgos y tratamiento	44
Trazabilidad	45
33. Roles de colaborador	45
Objetivo y decisión	45
Estado actual comprobable	45
Controles y procedimiento	45
Evidencia y criterio de gate	45
Riesgos y tratamiento	45
Trazabilidad	45
34. Acceso de proveedores	45
Objetivo y decisión	46
Estado actual comprobable	46
Controles y procedimiento	46
Evidencia y criterio de gate	46
Riesgos y tratamiento	46
Trazabilidad	46
35. Cuentas compartidas	46
Objetivo y decisión	47
Estado actual comprobable	47
Controles y procedimiento	47
Evidencia y criterio de gate	47
Riesgos y tratamiento	47
Trazabilidad	47
36. Password manager	47
Objetivo y decisión	47
Estado actual comprobable	48
Controles y procedimiento	48
Evidencia y criterio de gate	48
Riesgos y tratamiento	48
Trazabilidad	48
37. Rotación de secretos	48
Objetivo y decisión	48
Estado actual comprobable	48
Controles y procedimiento	48
Evidencia y criterio de gate	49
Riesgos y tratamiento	49
Trazabilidad	49
38. Secret scanning	49
Objetivo y decisión	49
Estado actual comprobable	49
Controles y procedimiento	49
Evidencia y criterio de gate	49

Riesgos y tratamiento	50
Trazabilidad	50
39. Variables de entorno	50
Objetivo y decisión	50
Estado actual comprobable	50
Controles y procedimiento	50
Evidencia y criterio de gate	50
Riesgos y tratamiento	50
Trazabilidad	51
40. Archivos VDF y SteamPipe	51
Objetivo y decisión	51
Estado actual comprobable	51
Controles y procedimiento	51
Evidencia y criterio de gate	51
Riesgos y tratamiento	51
Trazabilidad	52
41. API keys futuras	52
Objetivo y decisión	52
Estado actual comprobable	52
Controles y procedimiento	52
Evidencia y criterio de gate	52
Riesgos y tratamiento	52
Trazabilidad	52
42. Configuración segura por defecto	53
Objetivo y decisión	53
Estado actual comprobable	53
Controles y procedimiento	53
Evidencia y criterio de gate	53
Riesgos y tratamiento	53
Trazabilidad	53
43. Hardening de Windows de desarrollo	54
Objetivo y decisión	54
Estado actual comprobable	54
Controles y procedimiento	54
Evidencia y criterio de gate	54
Riesgos y tratamiento	54
Trazabilidad	54
44. Actualizaciones del sistema	54
Objetivo y decisión	54
Estado actual comprobable	55
Controles y procedimiento	55
Evidencia y criterio de gate	55
Riesgos y tratamiento	55
Trazabilidad	55
45. Antimalware y falsos positivos	55
Objetivo y decisión	55
Estado actual comprobable	56
Controles y procedimiento	56
Evidencia y criterio de gate	56
Riesgos y tratamiento	56
Trazabilidad	56
46. Cifrado de dispositivo	56
Objetivo y decisión	56
Estado actual comprobable	57
Controles y procedimiento	57
Evidencia y criterio de gate	57
Riesgos y tratamiento	57
Trazabilidad	57
47. Bloqueo y sesión	57

Objetivo y decisión	57
Estado actual comprobable	57
Controles y procedimiento	58
Evidencia y criterio de gate	58
Riesgos y tratamiento	58
Trazabilidad	58
48. Medios extraíbles	58
Objetivo y decisión	58
Estado actual comprobable	58
Controles y procedimiento	58
Evidencia y criterio de gate	59
Riesgos y tratamiento	59
Trazabilidad	59
49. Trabajo remoto	59
Objetivo y decisión	59
Estado actual comprobable	59
Controles y procedimiento	59
Evidencia y criterio de gate	59
Riesgos y tratamiento	60
Trazabilidad	60
50. Wi-Fi y red	60
Objetivo y decisión	60
Estado actual comprobable	60
Controles y procedimiento	60
Evidencia y criterio de gate	60
Riesgos y tratamiento	60
Trazabilidad	61
51. Descargas y fuentes confiables	61
Objetivo y decisión	61
Estado actual comprobable	61
Controles y procedimiento	61
Evidencia y criterio de gate	61
Riesgos y tratamiento	61
Trazabilidad	61
52. Macros y documentos	62
Objetivo y decisión	62
Estado actual comprobable	62
Controles y procedimiento	62
Evidencia y criterio de gate	62
Riesgos y tratamiento	62
Trazabilidad	62
53. Contenido generado y archivos externos	62
Objetivo y decisión	62
Estado actual comprobable	63
Controles y procedimiento	63
Evidencia y criterio de gate	63
Riesgos y tratamiento	63
Trazabilidad	63
54. Importación de Unity packages	63
Objetivo y decisión	63
Estado actual comprobable	63
Controles y procedimiento	64
Evidencia y criterio de gate	64
Riesgos y tratamiento	64
Trazabilidad	64
55. Editor scripts y ejecución automática	64
Objetivo y decisión	64
Estado actual comprobable	64
Controles y procedimiento	65

Evidencia y criterio de gate	65
Riesgos y tratamiento	65
Trazabilidad	65
56. Build scripts	65
Objetivo y decisión	65
Estado actual comprobable	65
Controles y procedimiento	65
Evidencia y criterio de gate	66
Riesgos y tratamiento	66
Trazabilidad	66
57. CI/CD futuro	66
Objetivo y decisión	66
Estado actual comprobable	66
Controles y procedimiento	66
Evidencia y criterio de gate	66
Riesgos y tratamiento	67
Trazabilidad	67
58. Protección de main	67
Objetivo y decisión	67
Estado actual comprobable	67
Controles y procedimiento	67
Evidencia y criterio de gate	67
Riesgos y tratamiento	67
Trazabilidad	68
59. Code review	68
Objetivo y decisión	68
Estado actual comprobable	68
Controles y procedimiento	68
Evidencia y criterio de gate	68
Riesgos y tratamiento	68
Trazabilidad	68
60. Change review de ProjectSettings	69
Objetivo y decisión	69
Estado actual comprobable	69
Controles y procedimiento	69
Evidencia y criterio de gate	69
Riesgos y tratamiento	69
Trazabilidad	69
61. Change review de Packages	69
Objetivo y decisión	69
Estado actual comprobable	70
Controles y procedimiento	70
Evidencia y criterio de gate	70
Riesgos y tratamiento	70
Trazabilidad	70
62. Dependency pinning	70
Objetivo y decisión	70
Estado actual comprobable	70
Controles y procedimiento	71
Evidencia y criterio de gate	71
Riesgos y tratamiento	71
Trazabilidad	71
63. Evaluación de actualizaciones	71
Objetivo y decisión	71
Estado actual comprobable	71
Controles y procedimiento	72
Evidencia y criterio de gate	72
Riesgos y tratamiento	72
Trazabilidad	72

64. Vulnerabilidades conocidas	72
Objetivo y decisión	72
Estado actual comprobable	72
Controles y procedimiento	72
Evidencia y criterio de gate	73
Riesgos y tratamiento	73
Trazabilidad	73
65. CVE y advisories	73
Objetivo y decisión	73
Estado actual comprobable	73
Controles y procedimiento	73
Evidencia y criterio de gate	73
Riesgos y tratamiento	74
Trazabilidad	74
66. Política de parcheo	74
Objetivo y decisión	74
Estado actual comprobable	74
Controles y procedimiento	74
Evidencia y criterio de gate	74
Riesgos y tratamiento	75
Trazabilidad	75
67. Ventanas de mantenimiento	75
Objetivo y decisión	75
Estado actual comprobable	75
Controles y procedimiento	75
Evidencia y criterio de gate	75
Riesgos y tratamiento	75
Trazabilidad	76
68. Excepciones de seguridad	76
Objetivo y decisión	76
Estado actual comprobable	76
Controles y procedimiento	76
Evidencia y criterio de gate	76
Riesgos y tratamiento	76
Trazabilidad	76
69. Aceptación de riesgo	77
Objetivo y decisión	77
Estado actual comprobable	77
Controles y procedimiento	77
Evidencia y criterio de gate	77
Riesgos y tratamiento	77
Trazabilidad	77
70. Secure coding C#	77
Objetivo y decisión	77
Estado actual comprobable	78
Controles y procedimiento	78
Evidencia y criterio de gate	78
Riesgos y tratamiento	78
Trazabilidad	78
71. Validación de entrada	78
Objetivo y decisión	78
Estado actual comprobable	78
Controles y procedimiento	78
Evidencia y criterio de gate	79
Riesgos y tratamiento	79
Trazabilidad	79
72. Paths y traversal	79
Objetivo y decisión	79
Estado actual comprobable	79

Controles y procedimiento	79
Evidencia y criterio de gate	79
Riesgos y tratamiento	80
Trazabilidad	80
73. Deserialización y JSON	80
Objetivo y decisión	80
Estado actual comprobable	80
Controles y procedimiento	80
Evidencia y criterio de gate	80
Riesgos y tratamiento	80
Trazabilidad	80
74. Archivos temporales	81
Objetivo y decisión	81
Estado actual comprobable	81
Controles y procedimiento	81
Evidencia y criterio de gate	81
Riesgos y tratamiento	81
Trazabilidad	81
75. Atomicidad de saves	81
Objetivo y decisión	81
Estado actual comprobable	82
Controles y procedimiento	82
Evidencia y criterio de gate	82
Riesgos y tratamiento	82
Trazabilidad	82
76. Corrupción y recuperación	82
Objetivo y decisión	82
Estado actual comprobable	82
Controles y procedimiento	82
Evidencia y criterio de gate	83
Riesgos y tratamiento	83
Trazabilidad	83
77. Integer overflow y economía	83
Objetivo y decisión	83
Estado actual comprobable	83
Controles y procedimiento	83
Evidencia y criterio de gate	84
Riesgos y tratamiento	84
Trazabilidad	84
78. Idempotencia	84
Objetivo y decisión	84
Estado actual comprobable	84
Controles y procedimiento	84
Evidencia y criterio de gate	84
Riesgos y tratamiento	85
Trazabilidad	85
79. Excepciones y fail-safe	85
Objetivo y decisión	85
Estado actual comprobable	85
Controles y procedimiento	85
Evidencia y criterio de gate	85
Riesgos y tratamiento	85
Trazabilidad	86
80. Logging seguro	86
Objetivo y decisión	86
Estado actual comprobable	86
Controles y procedimiento	86
Evidencia y criterio de gate	86
Riesgos y tratamiento	86

	Trazabilidad	86
81.	Redacción de logs	86
	Objetivo y decisión	87
	Estado actual comprobable	87
	Controles y procedimiento	87
	Evidencia y criterio de gate	87
	Riesgos y tratamiento	87
	Trazabilidad	87
82.	Dumps y datos sensibles	87
	Objetivo y decisión	87
	Estado actual comprobable	88
	Controles y procedimiento	88
	Evidencia y criterio de gate	88
	Riesgos y tratamiento	88
	Trazabilidad	88
83.	Network code futuro	88
	Objetivo y decisión	88
	Estado actual comprobable	88
	Controles y procedimiento	88
	Evidencia y criterio de gate	89
	Riesgos y tratamiento	89
	Trazabilidad	89
84.	TLS y certificados	89
	Objetivo y decisión	89
	Estado actual comprobable	89
	Controles y procedimiento	89
	Evidencia y criterio de gate	90
	Riesgos y tratamiento	90
	Trazabilidad	90
85.	Backend futuro	90
	Objetivo y decisión	90
	Estado actual comprobable	90
	Controles y procedimiento	90
	Evidencia y criterio de gate	90
	Riesgos y tratamiento	91
	Trazabilidad	91
86.	Autenticación futura	91
	Objetivo y decisión	91
	Estado actual comprobable	91
	Controles y procedimiento	91
	Evidencia y criterio de gate	91
	Riesgos y tratamiento	91
	Trazabilidad	92
87.	Autorización futura	92
	Objetivo y decisión	92
	Estado actual comprobable	92
	Controles y procedimiento	92
	Evidencia y criterio de gate	92
	Riesgos y tratamiento	92
	Trazabilidad	92
88.	Steam ID futuro	93
	Objetivo y decisión	93
	Estado actual comprobable	93
	Controles y procedimiento	93
	Evidencia y criterio de gate	93
	Riesgos y tratamiento	93
	Trazabilidad	93
89.	Anti-cheat y fraude	93
	Objetivo y decisión	93

Estado actual comprobable	94
Controles y procedimiento	94
Evidencia y criterio de gate	94
Riesgos y tratamiento	94
Trazabilidad	94
90. Manipulación de saves	94
Objetivo y decisión	94
Estado actual comprobable	94
Controles y procedimiento	94
Evidencia y criterio de gate	95
Riesgos y tratamiento	95
Trazabilidad	95
91. Integridad de economía offline	95
Objetivo y decisión	95
Estado actual comprobable	95
Controles y procedimiento	95
Evidencia y criterio de gate	96
Riesgos y tratamiento	96
Trazabilidad	96
92. Modding y Workshop futuros	96
Objetivo y decisión	96
Estado actual comprobable	96
Controles y procedimiento	96
Evidencia y criterio de gate	97
Riesgos y tratamiento	97
Trazabilidad	97
93. Plugins nativos	97
Objetivo y decisión	97
Estado actual comprobable	97
Controles y procedimiento	97
Evidencia y criterio de gate	97
Riesgos y tratamiento	98
Trazabilidad	98
94. Native DLLs	98
Objetivo y decisión	98
Estado actual comprobable	98
Controles y procedimiento	98
Evidencia y criterio de gate	98
Riesgos y tratamiento	98
Trazabilidad	99
95. Signing de ejecutables	99
Objetivo y decisión	99
Estado actual comprobable	99
Controles y procedimiento	99
Evidencia y criterio de gate	99
Riesgos y tratamiento	99
Trazabilidad	99
96. Reputación y SmartScreen	99
Objetivo y decisión	100
Estado actual comprobable	100
Controles y procedimiento	100
Evidencia y criterio de gate	100
Riesgos y tratamiento	100
Trazabilidad	100
97. Installer y distribución	100
Objetivo y decisión	100
Estado actual comprobable	101
Controles y procedimiento	101
Evidencia y criterio de gate	101

Riesgos y tratamiento	101
Trazabilidad	101
98. Steam depots y branches	101
Objetivo y decisión	101
Estado actual comprobable	101
Controles y procedimiento	102
Evidencia y criterio de gate	102
Riesgos y tratamiento	102
Trazabilidad	102
99. Rollback seguro	102
Objetivo y decisión	102
Estado actual comprobable	102
Controles y procedimiento	103
Evidencia y criterio de gate	103
Riesgos y tratamiento	103
Trazabilidad	103
100. Previous stable	103
Objetivo y decisión	103
Estado actual comprobable	103
Controles y procedimiento	103
Evidencia y criterio de gate	104
Riesgos y tratamiento	104
Trazabilidad	104
101. Release candidate	104
Objetivo y decisión	104
Estado actual comprobable	104
Controles y procedimiento	104
Evidencia y criterio de gate	104
Riesgos y tratamiento	105
Trazabilidad	105
102. Reproducibilidad	105
Objetivo y decisión	105
Estado actual comprobable	105
Controles y procedimiento	105
Evidencia y criterio de gate	105
Riesgos y tratamiento	105
Trazabilidad	106
103. Hash manifest	106
Objetivo y decisión	106
Estado actual comprobable	106
Controles y procedimiento	106
Evidencia y criterio de gate	106
Riesgos y tratamiento	106
Trazabilidad	106
104. THIRD_PARTY_NOTICES	107
Objetivo y decisión	107
Estado actual comprobable	107
Controles y procedimiento	107
Evidencia y criterio de gate	107
Riesgos y tratamiento	107
Trazabilidad	107
105. Security test plan	107
Objetivo y decisión	107
Estado actual comprobable	108
Controles y procedimiento	108
Evidencia y criterio de gate	108
Riesgos y tratamiento	108
Trazabilidad	108
106. Static analysis	108

Objetivo y decisión	108
Estado actual comprobable	108
Controles y procedimiento	109
Evidencia y criterio de gate	109
Riesgos y tratamiento	109
Trazabilidad	109
107. Dependency scanning	109
Objetivo y decisión	109
Estado actual comprobable	109
Controles y procedimiento	109
Evidencia y criterio de gate	110
Riesgos y tratamiento	110
Trazabilidad	110
108. Secret scanning automatizado	110
Objetivo y decisión	110
Estado actual comprobable	110
Controles y procedimiento	110
Evidencia y criterio de gate	111
Riesgos y tratamiento	111
Trazabilidad	111
109. Malware scanning de artefactos	111
Objetivo y decisión	111
Estado actual comprobable	111
Controles y procedimiento	111
Evidencia y criterio de gate	112
Riesgos y tratamiento	112
Trazabilidad	112
110. Fuzzing selectivo	112
Objetivo y decisión	112
Estado actual comprobable	112
Controles y procedimiento	112
Evidencia y criterio de gate	112
Riesgos y tratamiento	113
Trazabilidad	113
111. Tests de paths y archivos	113
Objetivo y decisión	113
Estado actual comprobable	113
Controles y procedimiento	113
Evidencia y criterio de gate	113
Riesgos y tratamiento	113
Trazabilidad	114
112. Tests de corrupción	114
Objetivo y decisión	114
Estado actual comprobable	114
Controles y procedimiento	114
Evidencia y criterio de gate	114
Riesgos y tratamiento	114
Trazabilidad	115
113. Tests de rollback	115
Objetivo y decisión	115
Estado actual comprobable	115
Controles y procedimiento	115
Evidencia y criterio de gate	115
Riesgos y tratamiento	115
Trazabilidad	116
114. Tests de permisos	116
Objetivo y decisión	116
Estado actual comprobable	116
Controles y procedimiento	116

Evidencia y criterio de gate	116
Riesgos y tratamiento	116
Trazabilidad	116
115. Tests en usuario estándar	117
Objetivo y decisión	117
Estado actual comprobable	117
Controles y procedimiento	117
Evidencia y criterio de gate	117
Riesgos y tratamiento	117
Trazabilidad	117
116. Clean machine test	117
Objetivo y decisión	117
Estado actual comprobable	118
Controles y procedimiento	118
Evidencia y criterio de gate	118
Riesgos y tratamiento	118
Trazabilidad	118
117. Proton/Deck security review	118
Objetivo y decisión	118
Estado actual comprobable	118
Controles y procedimiento	119
Evidencia y criterio de gate	119
Riesgos y tratamiento	119
Trazabilidad	119
118. Incident taxonomy	119
Objetivo y decisión	119
Estado actual comprobable	119
Controles y procedimiento	119
Evidencia y criterio de gate	120
Riesgos y tratamiento	120
Trazabilidad	120
119. Severidades SEC-0 a SEC-4	120
Objetivo y decisión	120
Estado actual comprobable	120
Controles y procedimiento	120
Evidencia y criterio de gate	120
Riesgos y tratamiento	121
Trazabilidad	121
120. Detección	121
Objetivo y decisión	121
Estado actual comprobable	121
Controles y procedimiento	121
Evidencia y criterio de gate	121
Riesgos y tratamiento	121
Trazabilidad	122
121. Canales de reporte	122
Objetivo y decisión	122
Estado actual comprobable	122
Controles y procedimiento	122
Evidencia y criterio de gate	122
Riesgos y tratamiento	122
Trazabilidad	122
122. Vulnerability disclosure	122
Objetivo y decisión	123
Estado actual comprobable	123
Controles y procedimiento	123
Evidencia y criterio de gate	123
Riesgos y tratamiento	123
Trazabilidad	123

123. Recepción responsable de reportes	123
Objetivo y decisión	123
Estado actual comprobable	124
Controles y procedimiento	124
Evidencia y criterio de gate	124
Riesgos y tratamiento	124
Trazabilidad	124
124. Triage de vulnerabilidad	124
Objetivo y decisión	124
Estado actual comprobable	124
Controles y procedimiento	124
Evidencia y criterio de gate	125
Riesgos y tratamiento	125
Trazabilidad	125
125. Contención	125
Objetivo y decisión	125
Estado actual comprobable	125
Controles y procedimiento	125
Evidencia y criterio de gate	126
Riesgos y tratamiento	126
Trazabilidad	126
126. Erradicación	126
Objetivo y decisión	126
Estado actual comprobable	126
Controles y procedimiento	126
Evidencia y criterio de gate	126
Riesgos y tratamiento	127
Trazabilidad	127
127. Recuperación	127
Objetivo y decisión	127
Estado actual comprobable	127
Controles y procedimiento	127
Evidencia y criterio de gate	127
Riesgos y tratamiento	127
Trazabilidad	128
128. Post-incident review	128
Objetivo y decisión	128
Estado actual comprobable	128
Controles y procedimiento	128
Evidencia y criterio de gate	128
Riesgos y tratamiento	128
Trazabilidad	129
129. Cadena de custodia	129
Objetivo y decisión	129
Estado actual comprobable	129
Controles y procedimiento	129
Evidencia y criterio de gate	129
Riesgos y tratamiento	129
Trazabilidad	129
130. Preservación de evidencia	129
Objetivo y decisión	130
Estado actual comprobable	130
Controles y procedimiento	130
Evidencia y criterio de gate	130
Riesgos y tratamiento	130
Trazabilidad	130
131. Reloj y timeline	130
Objetivo y decisión	130
Estado actual comprobable	131

Controles y procedimiento	131
Evidencia y criterio de gate	131
Riesgos y tratamiento	131
Trazabilidad	131
132. Comunicación interna	131
Objetivo y decisión	131
Estado actual comprobable	131
Controles y procedimiento	131
Evidencia y criterio de gate	132
Riesgos y tratamiento	132
Trazabilidad	132
133. Comunicación a jugadores	132
Objetivo y decisión	132
Estado actual comprobable	132
Controles y procedimiento	132
Evidencia y criterio de gate	133
Riesgos y tratamiento	133
Trazabilidad	133
134. Coordinación con Valve	133
Objetivo y decisión	133
Estado actual comprobable	133
Controles y procedimiento	133
Evidencia y criterio de gate	133
Riesgos y tratamiento	134
Trazabilidad	134
135. Coordinación con proveedor	134
Objetivo y decisión	134
Estado actual comprobable	134
Controles y procedimiento	134
Evidencia y criterio de gate	134
Riesgos y tratamiento	134
Trazabilidad	135
136. Brecha de datos	135
Objetivo y decisión	135
Estado actual comprobable	135
Controles y procedimiento	135
Evidencia y criterio de gate	135
Riesgos y tratamiento	135
Trazabilidad	135
137. Credencial comprometida	135
Objetivo y decisión	136
Estado actual comprobable	136
Controles y procedimiento	136
Evidencia y criterio de gate	136
Riesgos y tratamiento	136
Trazabilidad	136
138. Repositorio comprometido	136
Objetivo y decisión	136
Estado actual comprobable	137
Controles y procedimiento	137
Evidencia y criterio de gate	137
Riesgos y tratamiento	137
Trazabilidad	137
139. Paquete malicioso	137
Objetivo y decisión	137
Estado actual comprobable	137
Controles y procedimiento	137
Evidencia y criterio de gate	138
Riesgos y tratamiento	138

Trazabilidad	138
140. Build alterada	138
Objetivo y decisión	138
Estado actual comprobable	138
Controles y procedimiento	138
Evidencia y criterio de gate	139
Riesgos y tratamiento	139
Trazabilidad	139
141. Malware o ransomware	139
Objetivo y decisión	139
Estado actual comprobable	139
Controles y procedimiento	139
Evidencia y criterio de gate	139
Riesgos y tratamiento	140
Trazabilidad	140
142. Pérdida de dispositivo	140
Objetivo y decisión	140
Estado actual comprobable	140
Controles y procedimiento	140
Evidencia y criterio de gate	140
Riesgos y tratamiento	140
Trazabilidad	141
143. Publicación accidental	141
Objetivo y decisión	141
Estado actual comprobable	141
Controles y procedimiento	141
Evidencia y criterio de gate	141
Riesgos y tratamiento	141
Trazabilidad	141
144. Secretos en commit	141
Objetivo y decisión	142
Estado actual comprobable	142
Controles y procedimiento	142
Evidencia y criterio de gate	142
Riesgos y tratamiento	142
Trazabilidad	142
145. Save exploit crítico	142
Objetivo y decisión	143
Estado actual comprobable	143
Controles y procedimiento	143
Evidencia y criterio de gate	143
Riesgos y tratamiento	143
Trazabilidad	143
146. Denegación de servicio futura	143
Objetivo y decisión	143
Estado actual comprobable	144
Controles y procedimiento	144
Evidencia y criterio de gate	144
Riesgos y tratamiento	144
Trazabilidad	144
147. Abuso de comunidad	144
Objetivo y decisión	144
Estado actual comprobable	144
Controles y procedimiento	144
Evidencia y criterio de gate	145
Riesgos y tratamiento	145
Trazabilidad	145
148. Plan de crisis	145
Objetivo y decisión	145

Estado actual comprobable	145
Controles y procedimiento	145
Evidencia y criterio de gate	146
Riesgos y tratamiento	146
Trazabilidad	146
149. Business continuity	146
Objetivo y decisión	146
Estado actual comprobable	146
Controles y procedimiento	146
Evidencia y criterio de gate	146
Riesgos y tratamiento	147
Trazabilidad	147
150. RTO y RPO	147
Objetivo y decisión	147
Estado actual comprobable	147
Controles y procedimiento	147
Evidencia y criterio de gate	147
Riesgos y tratamiento	147
Trazabilidad	148
151. Single-person dependency	148
Objetivo y decisión	148
Estado actual comprobable	148
Controles y procedimiento	148
Evidencia y criterio de gate	148
Riesgos y tratamiento	148
Trazabilidad	149
152. Recuperación de cuentas	149
Objetivo y decisión	149
Estado actual comprobable	149
Controles y procedimiento	149
Evidencia y criterio de gate	149
Riesgos y tratamiento	149
Trazabilidad	150
153. Contactos de emergencia	150
Objetivo y decisión	150
Estado actual comprobable	150
Controles y procedimiento	150
Evidencia y criterio de gate	150
Riesgos y tratamiento	150
Trazabilidad	150
154. Simulacros	151
Objetivo y decisión	151
Estado actual comprobable	151
Controles y procedimiento	151
Evidencia y criterio de gate	151
Riesgos y tratamiento	151
Trazabilidad	151
155. Métricas de seguridad	151
Objetivo y decisión	151
Estado actual comprobable	151
Controles y procedimiento	152
Evidencia y criterio de gate	152
Riesgos y tratamiento	152
Trazabilidad	152
156. Security debt	152
Objetivo y decisión	152
Estado actual comprobable	152
Controles y procedimiento	152
Evidencia y criterio de gate	153

Riesgos y tratamiento	153
Trazabilidad	153
157. Work packages	153
158. Gates	154
Gate de desarrollo seguro	154
Gate H6	154
Gate Steamworks	154
Gate de sistema online	154
159. Checklist pre-commit	154
160. Checklist pre-build	154
161. Checklist pre-release	155
162. Plantilla de incidente	155
163. Plantilla de vulnerability report	155
164. Plantilla de risk acceptance	156
165. RACI	156
166. Definition of Ready	156
167. Definition of Done	157
168. Riesgos abiertos	157
169. Glosario	157
170. Fuentes oficiales	158
171. Inventario histórico	158
Documentos consolidados actuales consultados	159
172. Historial de cambios	159
Próxima revisión obligatoria	159

22 Security and Incident Response Plan

Metadatos normativos v2.0

document_version: 2.0
project_version: 0.0.26
last_reviewed: 2026-07-17
status: APPROVED LIVING PLAN / CONTINUOUS REVIEW
lifecycle: LIVING DOCUMENT

27 — Security and Incident Response Plan

Proyecto: Cartridge & Cloud

Estudio / Owner: VRM Games / Blas Luis Rocha González

Tecnología: Unity 6.3 LTS 6000.3.18f1, URP 17.3.0, C# 9 / netstandard2.1

Plataforma objetivo: Windows x64 / PC Steam

Versión de aplicación observada: 0.0.26

Fecha de consolidación: 2026-07-01

Estado: OPERATIONAL PLAN / IMPLEMENTATION PARTIAL / PUBLIC RELEASE BLOCKED

Este documento integra la genealogía histórica y el estado técnico observado. No es una certificación, un informe forense ni una garantía de ausencia de vulnerabilidades. Las capacidades futuras continúan NOT OPEN hasta superar sus gates.

Cómo leer los estados

- **IMPLEMENTED:** existe código o configuración.
- **VERIFIED:** existe prueba reproducible para la versión observada.
- **PARTIAL:** existe una parte, faltan controles o evidencia.

- PLANNED: política definida, ejecución pendiente.
- NOT OPEN: fuera de alcance y no autorizado.
- BLOCKED: no puede promoverse al gate indicado.

Resumen de postura

La inspección de 457 scripts C# no encontró llamadas de red, Steamworks, SDKs de analytics/crash reporting, DllImport, bloques unsafe ni patrones evidentes de secretos. Se localizaron 15 llamadas a File.Delete en 8 archivos y 7 llamadas a Directory.Delete en 7 archivos, principalmente en persistencia, recuperación y escenarios técnicos. El proyecto declara 41 dependencias directas y 57 entradas en packages-lock.json.

La seguridad actual se beneficia del alcance offline, la ausencia de código de red y una persistencia defensiva; sus principales brechas son operativas: identidad, backups, supply chain monitoring, repository controls, incident channel, candidate build hygiene y ejercicios de recuperación.

0. Propósito, alcance y autoridad

Propósito

Este documento establece el sistema de seguridad y respuesta a incidentes de **Cartridge & Cloud**. Protege el código, repositorio, documentación, arte fuente, paquetes, estaciones de trabajo, cuentas, secretos futuros, saves, builds, materiales de publicación y continuidad de VRM Games. Integra prevención, detección, respuesta, recuperación y aprendizaje.

Autoridad

La autoridad documental sigue la jerarquía vigente del proyecto. Este plan complementa —y no sustituye— 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 23_Legal_Credits_and_Licenses_Register.xlsx, 24_Steam_Publishing_Plan.md, 25_Post_Launch_and_Live y 26_Privacy_Data_and_Telemetry_Plan.md. Cuando una cuestión sea de privacidad, licencia o publicación, prevalece el documento especializado correspondiente y este plan aporta el procedimiento de seguridad.

Alcance

Incluye desarrollo local, Git/GitHub, Unity/UPM, Blender y herramientas DCC, backups, builds Windows, futura cuenta Steamworks, futura SteamPipe, soporte y vulnerabilidades reportadas. No afirma que el proyecto esté certificado contra NIST, OWASP, ISO 27001 u otro estándar. Los marcos se utilizan como vocabulario y checklist proporcional.

Objetivos de seguridad

1. Evitar pérdida irreversible del proyecto y de las credenciales.
2. Evitar publicación de una build manipulada, no autorizada o no reproducible.
3. Mantener la integridad de saves, economía e inventario frente a corrupción accidental.
4. Detectar secretos, dependencias vulnerables y cambios críticos antes del release.
5. Responder a incidentes con mínima improvisación y máxima preservación de evidencia.
6. Mantener sistemas online, Cloud y telemetría en NOT OPEN hasta superar sus gates.
7. Permitir que otra persona autorizada pueda recuperar el proyecto en una emergencia.

Estado del documento

OPERATIONAL PLAN / IMPLEMENTATION PARTIAL. La creación de este archivo no cierra automáticamente work packages, riesgos ni H6. El documento se revalida antes de onboarding Steamworks,

antes de la primera candidata pública, después de un incidente severo y cuando cambie el modelo de distribución o datos.

1. Genealogía histórica de seguridad

La seguridad del proyecto no nació como documento único. Se distribuyó en reglas de build, QA, legal, Steam, persistencia, handoff y ADR. Esta consolidación conserva esa genealogía y evita atribuir a versiones antiguas capacidades que todavía no existían.

Periodo	Fuente	Aportación de seguridad
v0.3	QA Testing Plan v0.3	Incluyó backup, archivo truncado, versión antigua y build candidata co
v0.3	Legal Register v0.3	Prohibió introducir recursos en build pública sin origen, licencia, uso co
v0.4	Build & Versioning v0.4	Añadió evidencia de las primeras builds Windows, checksum y pre-rele
v0.5	Package Manifest v0.3	Declaró que los paquetes documentales no incluían credenciales ni dat
v0.5	Build & Versioning v0.5	Separó builds locales de ZIP/checksum de hitos y mantuvo Buil
v0.6	Build & Versioning v0.6	Mantuvo 0.0.26, congeló tags/releases salvo gate y dejó StoreInitial pe
Sprint 14	ADR-0056 a ADR-0060	Formalizaron save v2 compatible, escritura validada, checksum/genera
Sprint 15	ADR-0063 a ADR-0066	Añadieron autosave idempotente, preferencias globales y exclusividad
Sprint 16	ADR-0069	Coordinó el checkpoint/autosave de Phase 1.

Regla de consolidación

- Una política histórica se conserva aunque hoy esté ampliada.
- Una build PASS histórica demuestra su gate, no la seguridad de la working copy actual.
- Los checksums históricos prueban integridad del artefacto archivado, no autenticidad criptográfica del productor.
- La ausencia histórica de Steamworks, backend o Cloud se mantiene como estado, no como deuda implícita que deba abrirse.
- Los cambios de v0.3 a v0.6 se tratan como evolución de preproducción a integración; no se elimina ninguna fuente previa.

2. Estado de seguridad actual

Resumen ejecutivo

La inspección de 457 scripts C# no encontró llamadas de red, Steamworks, SDKs de analytics/crash reporting, DllImport, bloques unsafe ni patrones evidentes de secretos. Se localizaron 15 llamadas a File.Delete en 8 archivos y 7 llamadas a Directory.Delete en 7 archivos, principalmente en persistencia, recuperación y escenarios técnicos. El proyecto declara 41 dependencias directas y 57 entradas en packages-lock.json.

Área	Evidencia actual
Runtime de red	Cero coincidencias de UnityWebRequest, HttpClient, sockets o Steamworks en
Analytics/crash SDK	Cero coincidencias de proveedores externos; Unity Analytics y Cloud Diagnostics
Secretos	Cero patrones evidentes en el árbol inspeccionado; .env, keystores y JKS están
Interop nativo	Cero DllImport y cero bloques unsafe
Operaciones destructivas	15 File.Delete y 7 Directory.Delete
Paquetes	41 directos; 57 entradas lock; sin scoped registry personalizado demostrado
Build profile	Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity,
Repositorio	.gitignore/.gitattributes presentes
Steamworks	Sin App ID, SDK, SteamPipe o branch pública
Public release	0 builds autorizadas en el registro legal

Área

Evidencia actual

Interpretación

El perfil local-first reduce amenazas de servidor, cuentas de jugador y exposición de API, pero no elimina riesgos de supply chain, pérdida del único workstation, compromiso de GitHub/Steamworks, secretos futuros, manipulación de build, ransomware, publicación accidental o corrupción de saves. El mayor riesgo actual es operativo: dependencia de una sola persona, ausencia de restore drills, ausencia de security tooling remoto demostrado y gates futuros todavía abiertos.

Prioridades inmediatas

1. Crear SECURITY.md y un canal privado antes de cualquier beta pública.
2. Activar MFA y documentar recovery para GitHub, correo, Unity y futuro Steamworks.
3. Ejecutar secret scan del historial Git y configurar prevención en push cuando esté disponible.
4. Establecer backup 3-2-1 y probar restauración del repositorio, arte fuente y documentación.
5. Crear SBOM/package audit y proceso de advisories.
6. Retirar TestLab de una candidata y conectar StoreInitial antes del gate H6.
7. Ejecutar un tabletop de credencial comprometida y otro de build dañina.

3. Principios Secure by Design

1. **Responsabilidad del productor.** El jugador no debe cargar con configuraciones inseguras, procesos de recuperación imposibles o información engañosa.
2. **Seguro por defecto.** Servicios online, telemetría, Cloud, mods, Workshop y SDKs externos permanecen desactivados hasta aprobación explícita.
3. **Menor superficie.** Eliminar paquetes, herramientas, escenas y privilegios innecesarios es preferible a proteger complejidad que el producto no necesita.
4. **Mínimo privilegio.** Cada cuenta, colaborador, token y script recibe únicamente el permiso y la duración requeridos.
5. **Defensa en profundidad.** .gitignore, MFA, secret scanning, branch protection, backups y revocación cubren fallos distintos; ninguno sustituye a los demás.
6. **Fail-safe y atomicidad.** Ante error, no confirmar una transacción parcial, no reemplazar un save válido por uno inválido y no promover una build incierta.
7. **Trazabilidad.** Toda release y todo incidente deben poder relacionarse con commit, dependencias, build, hash, manifest, owner y decisión.
8. **Recuperación probada.** Backup sin restore test no se considera control cerrado.
9. **Transparencia.** Los estados NOT OPEN, BLOCKED, PARTIAL y PASS se comunican sin convertir intención en realidad.
10. **Aprendizaje.** Incidentes, near misses y findings actualizan controles, tests y documentación.

Estos principios se alinean de forma proporcional con NIST CSF 2.0, NIST SSDF y la orientación Secure by Design; no constituyen una certificación.

4. Gobernanza y responsabilidad

Owner y modelo de operación

El owner de seguridad es **Blas Luis Rocha González / VRM Games**. En una producción individual, el mismo owner puede programar, revisar y publicar, pero debe introducir controles compensatorios: ramas, pausa deliberada, diff limpio, test automatizado, build reproducible, checklist firmada y conservación de previous_stable.

Responsabilidades mínimas

- Mantener inventario de activos, cuentas, paquetes y secretos futuros.
- Revisar advisories y vulnerabilidades relevantes.
- Autorizar excepciones y aceptar riesgo residual por escrito.
- Declarar incident commander durante SEC-0/SEC-1.
- Separar evidencia técnica de comunicación pública.
- Solicitar apoyo externo legal, forense o técnico cuando la capacidad interna sea insuficiente.

Separación de funciones proporcional

Para una release pública, se recomienda que al menos una segunda persona de confianza revise el Go/No-Go, permisos Steamworks, store/build selection y plan de rollback. Si no existe segundo revisor, la release se retrasa o se usa una revisión externa puntual; no se considera equivalente una relectura inmediata del propio autor.

Revisión

El plan se revisa trimestralmente durante desarrollo activo, antes de onboarding Steamworks, tras cambios de cuenta/proveedor, antes de una candidata pública y después de todo incidente SEC-0 a SEC-2.

5. Modelo de amenazas

Actores considerados

- Malware oportunista, ransomware y phishing.
- Atacante que obtiene una cuenta de correo, GitHub, Unity o Steamworks.
- Dependencia, plugin, extensión o archivo DCC comprometido.
- Colaborador o proveedor con exceso de permisos.
- Jugador que manipula saves locales o entradas para provocar estados inválidos.
- Error humano: publicación accidental, borrado, merge incorrecto o filtración de secreto.
- Fallo de proveedor, almacenamiento, hardware o sincronización.

Objetivos del adversario o del fallo

Robar código/arte, insertar malware, distribuir una build alterada, secuestrar una cuenta, destruir backups, extraer datos de soporte, manipular economía, provocar corrupción, suplantar al estudio o extorsionar.

Supuestos

El runtime actual no posee backend ni datos de cuenta, por lo que el foco se coloca en desarrollo, supply chain, distribución y saves. Este modelo cambia de forma sustancial al abrir Steam Cloud, telemetría, cuentas, Workshop, web propia o servicios de soporte.

Método

Cada nueva feature identifica activos, entradas, trust boundaries, abuso, impacto, controles, detección, recuperación y owner. El resultado se enlaza al risk register; una amenaza no mitigada puede cerrar el gate aunque la feature funcione técnicamente.

6. Activos que proteger

Clase	Ejemplos	Impacto principal
Propiedad intelectual	Código, GDD, arte, audio, modelos, branding	Pérdida, filtración o dispersión
Fuente reproducible	Assets, Packages, ProjectSettings, Tools, documentación	Imposibilidad de reconstrucción
Identidad	Email, GitHub, Unity, Steamworks, dominio futuro	Suplantación y publicación
Secretos	Tokens, passwords, recovery codes, VDF con credenciales	Acceso persistente a cuentas
Builds	ZIP, ejecutable, depots, manifests, previous stable	Distribución de malware
Datos locales	Saves, backups, preferencias, logs	Pérdida de progreso o experiencia
Evidencia	Logs, hashes, screenshots, timeline, incident records	Incapacidad de investigación
Continuidad	Backups, contactos, instrucciones de restore	Parada del proyecto por falta de recursos

Cada activo recibe ID, owner, ubicación primaria, clasificación, dependencias, backup, RPO/RTO y gate de distribución. Los conceptos y assets VISION / NOT OPEN también se protegen para evitar publicación accidental, aunque no formen parte de la candidata.

7. Superficies de ataque

Las superficies actuales son el workstation Windows, navegador/correo, Git/GitHub, Unity Hub/Editor/UPM, Visual Studio, Blender, archivos descargados, almacenamiento de backups y canales de documentación. Las superficies futuras incluyen Steamworks, SteamPipe, soporte público, website, Cloud, analytics y crash reporting.

El runtime offline recibe inputs de usuario, archivos JSON locales, PlayerPrefs y assets empaquetados. Aunque no exponga sockets, debe soportar archivos corruptos, paths inesperados, interrupciones durante escritura y valores fuera de rango. El Editor posee más privilegios que el Player: editor scripts, importers y plugins se tratan como código ejecutable.

Toda apertura de superficie requiere owner, threat model, provider review, least privilege, logging mínimo, shutdown plan y test de recuperación.

8. Límites de confianza

1. **Internet → workstation:** descargas, email, navegador, package registry y archivos externos no son confiables por defecto.
2. **Proveedor → repositorio:** un paquete oficial sigue siendo tercero y puede cambiar, quedar vulnerable o comprometerse.
3. **Editor → Player:** código/editor tooling no debe filtrarse a la build pública.
4. **Repositorio → build:** una build solo es confiable si procede de commit, dependencias y script identificados.
5. **Build → Steam:** upload y promoción requieren credencial, branch, manifest y decisión separadas.
6. **Jugador → save/log:** archivos locales pueden estar corruptos o manipulados; soporte no debe tratarlos como datos confiables ni seguros.
7. **Canal público → incidente:** un reporte puede ser genuino, incompleto, malicioso o contener datos personales; se preserva y valida.

Los límites se documentan en arquitectura y se revisan al añadir una integración o cambiar el flujo de datos.

9. Clasificación de información

Nivel	Ejemplos	Tratamiento
PUBLIC	Store copy aprobado, patch notes, press kit	Publicable tras revisión
INTERNAL	Roadmap, QA, documentación de trabajo	Compartición limitada al proyecto

Nivel	Ejemplos	Tratamiento
CONFIDENTIAL	Build no anunciada, contratos, incidentes abiertos	Acceso por necesidad
SECRET	Passwords, tokens, recovery codes, claves de firma	Password manager/HSM; num
PERSONAL/RESTRICTED	Datos fiscales, tickets, saves/logs de jugadores	Plan de Privacidad, minimiza

La clasificación acompaña al activo, no al formato. Una captura puede contener un token; un log puede contener una ruta de usuario; un VDF puede ser una plantilla inocua o incluir credenciales. Antes de compartir se revisa contenido efectivo, metadatos y audiencia.

10. Código fuente

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Código fuente**, la decisión por defecto es **PARTIAL / DOCUMENTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: El inventario contiene 457 scripts C# y 12 asmdefs; la suite aceptada es 1215 EditMode + 70 PlayMode. No se ha demostrado revisión de seguridad independiente.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Revisar cambios de persistencia, paths, economía, serialización, editor scripts y composición runtime con checklist específica.
6. Para trabajo individual, compensar la falta de segundo revisor con branch, diff limpio, tests, pausa deliberada y revisión posterior.

Evidencia y criterio de gate

- Diff, commit, test run y checklist de revisión.
- ADR cuando cambia una invariante o trust boundary.

Criterio PASS de Código fuente: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Normalizar revisión superficial por ser un proyecto individual.
- Introducir una regresión de integridad en un cambio aparentemente local.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

11. Repositorios y ramas

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Repositorios y ramas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta .gitignore y .gitattributes, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o SECURITY.md configurados en el repositorio remoto.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Proteger main, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.

Criterio PASS de Repositorios y ramas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

12. Credenciales y secretos

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Credenciales y secretos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Credenciales y secretos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

13. Steamworks y cuentas de partner

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Steamworks y cuentas de partner**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Steamworks y cuentas de partner: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

14. Unity ID y servicios

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Unity ID y servicios**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.

3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Unity ID y servicios: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir .gitignore con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

15. GitHub y dispositivos

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **GitHub y dispositivos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta .gitignore y .gitattributes, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o SECURITY.md configurados en el repositorio remoto.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Proteger main, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.

Criterio PASS de GitHub y dispositivos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

16. Estaciones de trabajo

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Estaciones de trabajo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Estaciones de trabajo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

17. Blender y herramientas DCC

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Blender y herramientas DCC**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes .blend y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.
6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Blender y herramientas DCC: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.
- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

18. Paquetes y dependencias

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Paquetes y dependencias**, la decisión por defecto es **PARTIAL / DOCUMENTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Paquetes y dependencias: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

19. Supply chain de Unity Package Manager

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Supply chain de Unity Package Manager**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Supply chain de Unity Package Manager: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

20. SBOM

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **SBOM**, la decisión por defecto es

PLANNED: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de SBOM: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

21. Licencias y seguridad

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Licencias y seguridad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El registro legal mantiene revisión de LICENSE/NOTICE pendiente por paquete y bloquea distribución pública mientras no se cierre.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Tratar licencia, procedencia y vulnerabilidad como dimensiones separadas.
6. Conservar texto legal exacto por versión y actualizar notices al cambiar paquetes.

Evidencia y criterio de gate

- Registro 23 y archivo de notices generado para la candidata.
- Hash y versión de cada dependencia distribuida.

Criterio PASS de Licencias y seguridad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Distribuir software sin cumplir avisos.
- Confundir paquete oficial con licencia automáticamente resuelta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

22. Artefactos de build

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Artefactos de build**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Artefactos de build: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

23. Integridad y checksums

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Integridad y checksums**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Integridad y checksums: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

24. Firmas y procedencia

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Firmas y procedencia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Firmas y procedencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

25. Backups del repositorio

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Backups del repositorio**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Backups del repositorio: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

26. Backups documentales

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Backups documentales**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Backups documentales: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

27. Backups de herramientas y fuentes

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Backups de herramientas y fuentes**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Backups de herramientas y fuentes: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

28. Recuperación ante pérdida total

Objetivo y decisión

Este capítulo protege un activo concreto y define su owner, ubicación, clasificación, dependencia, copia de recuperación y condiciones de distribución. Para **Recuperación ante pérdida total**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario. Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Inventariar ubicación primaria, copias, sensibilidad, dependencias y responsable.
2. Aplicar control de acceso mínimo y evitar copias no trazadas.
3. Conservar una copia recuperable fuera del fallo común del activo primario.
4. Probar restauración, no solo existencia del backup.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.
9. Preservar copia antes de modificar el sistema afectado.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.
- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Recuperación ante pérdida total: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.
- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

29. Control de acceso

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Control de acceso**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Control de acceso: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

30. Principio de mínimo privilegio

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Principio de mínimo privilegio**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Principio de mínimo privilegio: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

31. MFA

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **MFA**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.git-ignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de MFA: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir .gitignore con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

32. Altas, bajas y cambios de acceso

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Altas, bajas y cambios de acceso**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Altas, bajas y cambios de acceso: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

33. Roles de colaborador

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Roles de colaborador**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.

Criterio PASS de Roles de colaborador: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

34. Acceso de proveedores

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Acceso de proveedores**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es principalmente individual y no se ha demostrado una matriz operativa de accesos externos. Steamworks todavía no existe como cuenta de partner activa. No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Conceder acceso por función y duración, no por comodidad.
6. Revisar y retirar permisos al terminar colaboración o soporte.
7. Separar cuentas de publicación, desarrollo y soporte cuando el proveedor lo permita.
8. Separar facts, hipótesis, impacto, acciones y próxima actualización.
9. Coordinar mensajes técnicos, legales, privacidad y comunidad.

Evidencia y criterio de gate

- RACI y registro de accesos.
- Comprobación de baja y revocación de tokens.
- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Acceso de proveedores: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Acceso persistente de un colaborador antiguo.
- Cuenta con permiso de publicación comprometida.
- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

35. Cuentas compartidas

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Cuentas compartidas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Cuentas compartidas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

36. Password manager

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Password manager**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Password manager: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

37. Rotación de secretos

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Rotación de secretos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.

3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Rotación de secretos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir .gitignore con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

38. Secret scanning

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Secret scanning**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera PLANNED hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Secret scanning: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

39. Variables de entorno

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Variables de entorno**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. .gitignore excluye .env, .env.*, *.keystore y *.jks; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de Variables de entorno: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir .gitignore con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

40. Archivos VDF y SteamPipe

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **Archivos VDF y SteamPipe**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. .gitignore excluye .env, .env.*, *.keystore y *.jks; Steamworks y SteamPipe no están integrados. EditorBuildSettings.asset incluye Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity, Assets/_Project/Scenes/Store.unity, Assets/_Project/Scenes/TestLab.unity. StoreInitial no está en el build profile y TestLab sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.
8. Generar build desde script reproducible y lista de escenas explícita.
9. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.
- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Archivos VDF y SteamPipe: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir .gitignore con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.
- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

41. API keys futuras

Objetivo y decisión

Este capítulo gobierna identidades, permisos y secretos para reducir compromiso de cuentas, fuga de credenciales y acciones no atribuibles. Para **API keys futuras**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. .git-ignore excluye .env, .env.*, *.keystore y *.jks; Steamworks y SteamPipe no están integrados.

Controles y procedimiento

1. Utilizar identidad individual; prohibir cuentas compartidas salvo excepción documentada.
2. Activar MFA resistente al phishing cuando el proveedor lo permita.
3. Guardar recuperación y secretos en un password manager, nunca en el repositorio.
4. Revisar permisos después de cambios de rol, colaboración o incidente.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.

Criterio PASS de API keys futuras: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir .gitignore con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

42. Configuración segura por defecto

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Configuración segura por defecto**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Package Manager tiene preview packages desactivados y registry oficial; Version Control usa Visible Meta Files. UnityConnectSettings desactiva Analytics y Cloud Diagnostics, pero mantiene EngineDiagnosticsEnabled: 1; ProjectSettings.asset contiene submitAnalytics: 1, por lo que debe auditarse el comportamiento real sin asumir transmisión. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar diffs de ProjectSettings y Packages como cambios de infraestructura.
6. Documentar settings ambiguos y validar con build/red observada cuando sean relevantes.
7. No activar servicios cloud o diagnostics por accidente al abrir el Editor.
8. Usar IDs estables, owner, target, evidencia y estado.
9. Cerrar deuda solo con prueba; una aceptación necesita vencimiento y riesgo residual.

Evidencia y criterio de gate

- Snapshot de settings y prueba de red/servicio si se abre el gate.
- Diff aprobado y resultado de build.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Configuración segura por defecto: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Activar recopilación o servicio no documentado.
- Cambiar escenas, identificador o build flags sin revisión.
- Convertir la documentación en cumplimiento de papel.
- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

43. Hardening de Windows de desarrollo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Hardening de Windows de desarrollo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Hardening de Windows de desarrollo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

44. Actualizaciones del sistema

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Actualizaciones del sistema**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera

cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas. `manifest.json` declara 41 dependencias directas y `packages-lock.json` 57 entradas. Entre los paquetes directos figura `com.unity.localization` 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.
9. Versionar manifest y lockfile; revisar cambios directos y transitivos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.
- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Actualizaciones del sistema: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.
- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

45. Antimalware y falsos positivos

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Antimalware**

y **falsos positivos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas. El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.
9. Detener propagación y acceso sin borrar evidencia.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.
- Scope confirmado, loCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Antimalware y falsos positivos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

46. Cifrado de dispositivo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Cifrado de dispositivo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera

cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Cifrado de dispositivo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

47. Bloqueo y sesión

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Bloqueo y sesión**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Bloqueo y sesión: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

48. Medios extraíbles

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Medios extraíbles**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.

7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Medios extraíbles: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

49. Trabajo remoto

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Trabajo remoto**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Trabajo remoto: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

50. Wi-Fi y red

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Wi-Fi y red**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se aportó una baseline verificable de hardening del equipo de desarrollo, cifrado, antivirus, backups de dispositivo o cuentas administrativas.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Usar cuenta diaria sin privilegios administrativos cuando sea viable.
6. Mantener sistema, navegador, Unity Hub, IDE, Git y Blender actualizados por riesgo.
7. Activar cifrado de disco, bloqueo automático y recuperación protegida.
8. Escanear medios y archivos externos antes de abrirlos.

Evidencia y criterio de gate

- Checklist fechada de workstation y versión de herramientas.
- Prueba de backup/restauración y de recuperación de cuenta.

Criterio PASS de Wi-Fi y red: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ransomware o robo del dispositivo como fallo único.
- Instalar extensiones o herramientas manipuladas.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

51. Descargas y fuentes confiables

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Descargas y fuentes confiables**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes .blend y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.
6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Descargas y fuentes confiables: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.
- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

52. Macros y documentos

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Macros y documentos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes .blend y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.
6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Macros y documentos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.
- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

53. Contenido generado y archivos externos

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Contenido generado y archivos externos**, la decisión por defecto es **PLANNED**: ninguna afirmación de

seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen herramientas y conceptos generados, además de fuentes .blend y modelos; no se ha demostrado sandbox o revisión automática de scripts/macros embebidos.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Abrir archivos externos en una copia y revisar scripts, drivers, plugins y macros antes de confiar en ellos.
6. Conservar fuente, hash, procedencia y versión de herramienta.
7. No ejecutar archivos descargados con privilegios elevados.

Evidencia y criterio de gate

- Registro de procedencia y hash del archivo original.
- Resultado de escaneo y revisión de scripts/plugins.

Criterio PASS de Contenido generado y archivos externos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Ejecución de código desde un asset o plugin.
- Contaminar el proyecto con una fuente de procedencia ambigua.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

54. Importación de Unity packages

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Importación de Unity packages**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test

Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Importación de Unity packages: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

55. Editor scripts y ejecución automática

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Editor scripts y ejecución automática**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Editor scripts y ejecución automática: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

56. Build scripts

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Build scripts**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

EditorBuildSettings.asset incluye Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity, Assets/_Project/Scenes/Store.unity, Assets/_Project/Scenes/TestLab.unity. StoreInitial no está en el build profile y TestLab sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.

7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Build scripts: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

57. CI/CD futuro

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **CI/CD futuro**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainM`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración `SteamPipe` demostrada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir `TestLab`, herramientas `Editor`, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de CI/CD futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

58. Protección de main

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Protección de main**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta .gitignore y .gitattributes, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o SECURITY.md configurados en el repositorio remoto.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Proteger main, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.

Criterio PASS de Protección de main: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

59. Code review

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Code review**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: El inventario contiene 457 scripts C# y 12 asmdefs; la suite aceptada es 1215 EditMode + 70 PlayMode. No se ha demostrado revisión de seguridad independiente.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar cambios de persistencia, paths, economía, serialización, editor scripts y composición runtime con checklist específica.
6. Para trabajo individual, compensar la falta de segundo revisor con branch, diff limpio, tests, pausa deliberada y revisión posterior.

Evidencia y criterio de gate

- Diff, commit, test run y checklist de revisión.
- ADR cuando cambia una invariante o trust boundary.

Criterio PASS de Code review: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Normalizar revisión superficial por ser un proyecto individual.
- Introducir una regresión de integridad en un cambio aparentemente local.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

60. Change review de ProjectSettings

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Change review de ProjectSettings**, la decisión por defecto es **PARTIAL / DOCUMENTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Package Manager tiene preview packages desactivados y registry oficial; Version Control usa Visible Meta Files. UnityConnectSettings desactiva Analytics y Cloud Diagnostics, pero mantiene EngineDiagnosticsEnabled: 1; ProjectSettings.asset contiene submitAnalytics: 1, por lo que debe auditarse el comportamiento real sin asumir transmisión.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar diffs de ProjectSettings y Packages como cambios de infraestructura.
6. Documentar settings ambiguos y validar con build/red observada cuando sean relevantes.
7. No activar servicios cloud o diagnostics por accidente al abrir el Editor.

Evidencia y criterio de gate

- Snapshot de settings y prueba de red/servicio si se abre el gate.
- Diff aprobado y resultado de build.

Criterio PASS de Change review de ProjectSettings: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Activar recopilación o servicio no documentado.
- Cambiar escenas, identificador o build flags sin revisión.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

61. Change review de Packages

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Change review de Packages**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera

cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Package Manager tiene preview packages desactivados y registry oficial; Version Control usa Visible Meta Files. UnityConnectSettings desactiva Analytics y Cloud Diagnostics, pero mantiene EngineDiagnosticsEnabled: 1; ProjectSettings.asset contiene submitAnalytics: 1, por lo que debe auditarse el comportamiento real sin asumir transmisión.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Revisar diffs de ProjectSettings y Packages como cambios de infraestructura.
6. Documentar settings ambiguos y validar con build/red observada cuando sean relevantes.
7. No activar servicios cloud o diagnostics por accidente al abrir el Editor.

Evidencia y criterio de gate

- Snapshot de settings y prueba de red/servicio si se abre el gate.
- Diff aprobado y resultado de build.

Criterio PASS de Change review de Packages: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Activar recopilación o servicio no documentado.
- Cambiar escenas, identificador o build flags sin revisión.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

62. Dependency pinning

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Dependency pinning**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test

Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Dependency pinning: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

63. Evaluación de actualizaciones

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Evaluación de actualizaciones**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Evaluación de actualizaciones: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

64. Vulnerabilidades conocidas

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Vulnerabilidades conocidas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera PLANNED hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Vulnerabilidades conocidas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

65. CVE y advisories

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **CVE y advisories**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de CVE y advisories: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

66. Política de parcheo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Política de parcheo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Política de parcheo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

67. Ventanas de mantenimiento

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Ventanas de mantenimiento**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera PLANNED hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Ventanas de mantenimiento: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

68. Excepciones de seguridad

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Excepciones de seguridad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Excepciones de seguridad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

69. Aceptación de riesgo

Objetivo y decisión

Este capítulo reduce la superficie de ataque de estaciones, herramientas, repositorio y cadena de suministro mediante configuración, inventario y mantenimiento controlado. Para **Aceptación de riesgo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera PLANNED hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Aplicar parches por riesgo y comprobar compatibilidad antes de promover cambios.
2. Descargar herramientas y paquetes únicamente de fuentes verificadas.
3. Mantener configuración mínima, eliminar componentes sin uso y registrar excepciones.
4. Conservar rollback de la herramienta o dependencia anterior durante la validación.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Aceptación de riesgo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

70. Secure coding C#

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Secure coding C#**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: El inventario contiene 457 scripts C# y 12 asmdefs; la suite aceptada es 1215 EditMode + 70 PlayMode. No se ha demostrado revisión de seguridad independiente.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infraestructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Revisar cambios de persistencia, paths, economía, serialización, editor scripts y composición runtime con checklist específica.
6. Para trabajo individual, compensar la falta de segundo revisor con branch, diff limpio, tests, pausa deliberada y revisión posterior.

Evidencia y criterio de gate

- Diff, commit, test run y checklist de revisión.
- ADR cuando cambia una invariante o trust boundary.

Criterio PASS de Secure coding C#: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Normalizar revisión superficial por ser un proyecto individual.
- Introducir una regresión de integridad en un cambio aparentemente local.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

71. Validación de entrada

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Validación de entrada**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infraestructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.

5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Validación de entrada: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

72. Paths y traversal

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Paths y traversal**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Paths y traversal: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

73. Deserialización y JSON

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Deserialización y JSON**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Deserialización y JSON: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

74. Archivos temporales

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Archivos temporales**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Archivos temporales: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

75. Atomicidad de saves

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Atomicidad de saves**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.

Criterio PASS de Atomicidad de saves: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

76. Corrupción y recuperación

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Corrupción y recuperación**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.

9. Preservar copia antes de modificar el sistema afectado.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Corrupción y recuperación: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

77. Integer overflow y economía

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Integer overflow y economía**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La economía usa unidades menores e invariantes atómicas; los ADR históricos protegen ledger, reservas, receiving, checkout y cierres diarios.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Validar rangos antes de sumar, multiplicar o convertir.
6. Usar identificadores idempotentes para operaciones repetibles.
7. No confiar en valores cargados hasta validarlos contra invariantes.

Evidencia y criterio de gate

- Tests de límites, repetición, rollback y ledger.
- Snapshot inválido rechazado sin mutación parcial.

Criterio PASS de Integer overflow y economía: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Desbordamiento, saldo negativo o duplicación.
- Repetición de una operación tras crash o retry.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

78. Idempotencia

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Idempotencia**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La economía usa unidades menores e invariantes atómicas; los ADR históricos protegen ledger, reservas, receiving, checkout y cierres diarios.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Validar rangos antes de sumar, multiplicar o convertir.
6. Usar identificadores idempotentes para operaciones repetibles.
7. No confiar en valores cargados hasta validarlos contra invariantes.

Evidencia y criterio de gate

- Tests de límites, repetición, rollback y ledger.
- Snapshot inválido rechazado sin mutación parcial.

Criterio PASS de Idempotencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Desbordamiento, saldo negativo o duplicación.
- Repetición de una operación tras crash o retry.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

79. Excepciones y fail-safe

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Excepciones y fail-safe**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Excepciones y fail-safe: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

80. Logging seguro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Logging seguro**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Logging seguro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 26_Privacy_Data_and_Telemetry_Plan.md.

81. Redacción de logs

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Redacción de logs**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Redacción de logs: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`, `26_Privacy_Data_and_Telemetry_Plan.md`.

82. Dumps y datos sensibles

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Dumps y datos sensibles**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existe `Player.log` y logging técnico, pero no un proveedor externo. El Plan de Privacidad prohíbe transmitir logs, dumps o saves sin inventario, minimización y canal controlado.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No incluir tokens, rutas completas, nombres de usuario, emails ni contenido de save en logs públicos.
6. Registrar contexto técnico suficiente sin datos innecesarios.
7. Diferenciar error recuperable, corrupción y fallo fatal.

Evidencia y criterio de gate

- Scan de logs y test de redacción.
- Ejemplo de incidente reproducible sin datos personales.

Criterio PASS de Dumps y datos sensibles: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Filtrar secretos o datos de jugador mediante soporte.
- Ocultar la causa raíz con mensajes genéricos sin correlation ID.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 26_Privacy_Data_and_Telemetry_Plan.md.

83. Network code futuro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Network code futuro**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece NOT OPEN.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.

3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Network code futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

84. TLS y certificados

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **TLS y certificados**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece NOT OPEN.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de TLS y certificados: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

85. Backend futuro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Backend futuro**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece NOT OPEN.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Backend futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

86. Autenticación futura

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Autenticación futura**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece NOT OPEN.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Autenticación futura: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

87. Autorización futura

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Autorización futura**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece NOT OPEN.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Autorización futura: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

88. Steam ID futuro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Steam ID futuro**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece NOT OPEN.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infraestructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Steam ID futuro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md, 26_Privacy_Data_and_Telemetry_Plan.md.

89. Anti-cheat y fraude

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Anti-cheat y fraude**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El juego es single-player offline; no existe economía real, competición, leaderboard ni Workshop. El save local puede ser modificado por el propietario del equipo.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No prometer inviolabilidad del save local.
6. Proteger invariantes y recuperación, no desplegar DRM/anti-cheat desproporcionado.
7. Reevaluar al introducir logros, rankings, Cloud, Workshop o contenido compartido.

Evidencia y criterio de gate

- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Anti-cheat y fraude: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Penalizar al jugador offline sin beneficio real.
- Reutilizar datos locales manipulables en un futuro sistema competitivo.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

90. Manipulación de saves

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Manipulación de saves**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.

7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. No prometer inviolabilidad del save local.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Manipulación de saves: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Penalizar al jugador offline sin beneficio real.
- Reutilizar datos locales manipulables en un futuro sistema competitivo.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

91. Integridad de economía offline

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Integridad de economía offline**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26. La economía usa unidades menores e invariantes atómicas; los ADR históricos protegen ledger, reservas, receiving, checkout y cierres diarios. El juego es single-player offline; no existe economía real, competición, leaderboard ni Workshop. El save local puede ser modificado por el propietario del equipo.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.

7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.
9. Validar rangos antes de sumar, multiplicar o convertir.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.
- Tests de límites, repetición, rollback y ledger.
- Snapshot inválido rechazado sin mutación parcial.
- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Integridad de economía offline: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.
- Desbordamiento, saldo negativo o duplicación.
- Repetición de una operación tras crash o retry.
- Penalizar al jugador offline sin beneficio real.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

92. Modding y Workshop futuros

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Modding y Workshop futuros**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El juego es single-player offline; no existe economía real, competición, leaderboard ni Workshop. El save local puede ser modificado por el propietario del equipo.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. No prometer inviolabilidad del save local.
6. Proteger invariantes y recuperación, no desplegar DRM/anti-cheat desproporcionado.
7. Reevaluar al introducir logros, rankings, Cloud, Workshop o contenido compartido.

Evidencia y criterio de gate

- Decisión de alcance y tests de save inválido.
- Threat model actualizado antes de abrir sistemas competitivos.

Criterio PASS de Modding y Workshop futuros: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Penalizar al jugador offline sin beneficio real.
- Reutilizar datos locales manipulables en un futuro sistema competitivo.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

93. Plugins nativos

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Plugins nativos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Plugins nativos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

94. Native DLLs

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Native DLLs**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.

Criterio PASS de Native DLLs: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

95. Signing de ejecutables

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Signing de ejecutables**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Signing de ejecutables: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

96. Reputación y SmartScreen

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Reputación y SmartScreen**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Reputación y SmartScreen: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

97. Installer y distribución

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Installer y distribución**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Installer y distribución: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

98. Steam depots y branches

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Steam depots y branches**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El paquete aporta .gitignore y .gitattributes, pero no demuestra reglas de protección de rama, Actions, Dependabot, secret scanning o SECURITY.md configurados en el repositorio remoto. EditorBuildSettings.asset incluye Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity, Assets/_Project/Scenes/Store.unity, Assets/_Project/Scenes/TestLab.un. StoreInitial no está en el build profile y TestLab sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Proteger main, exigir checks y evitar force-push en ramas de release cuando la plataforma lo permita.
6. Mantener una rama corta por cambio y promover únicamente tras validación.
7. Generar build desde script reproducible y lista de escenas explícita.
8. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
9. Promover una misma candidata validada; no recompilar después del Go.

Evidencia y criterio de gate

- Captura/export de settings de repositorio.
- Historial de PR o registro de revisión compensatoria.
- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Steam depots y branches: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Pérdida de historial por force-push.
- Promoción directa de código no probado o de una cuenta comprometida.
- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

99. Rollback seguro

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Rollback seguro**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

EditorBuildSettings.asset incluye Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity, Assets/_Project/Scenes/Store.unity, Assets/_Project/Scenes/TestLab.unity. StoreInitial no está en el build profile y TestLab sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar previous_stable y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Rollback seguro: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 25_Post_Launch_and_Live_Operations_Plan.md.

100. Previous stable

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Previous stable**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

EditorBuildSettings.asset incluye Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity, Assets/_Project/Scenes/Store.unity, Assets/_Project/Scenes/TestLab.unity. StoreInitial no está en el build profile y TestLab sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.

7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Previous stable: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 25_Post_Launch_and_Live_Operations_Plan.md.

101. Release candidate

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Release candidate**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

`EditorBuildSettings.asset` incluye `Assets/_Project/Scenes/Bootstrap.unity`, `Assets/_Project/Scenes/MainMenu.unity`, `Assets/_Project/Scenes/Store.unity`, `Assets/_Project/Scenes/TestLab.unity`. `StoreInitial` no está en el build profile y `TestLab` sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración `SteamPipe` demostrada.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir `TestLab`, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar `previous_stable` y probar rollback antes de publicar.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.

Criterio PASS de Release candidate: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

102. Reproducibilidad

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Reproducibilidad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Reproducibilidad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

103. Hash manifest

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **Hash manifest**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Las builds históricas registran hashes y ejecución externa; no existe firma de código Windows, pipeline de provenance o candidata Steam aprobada. La aplicación está en versión 0.0.26.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Construir desde commit limpio y dependencias bloqueadas.
6. Registrar versión, commit, Unity, manifest, escenas, checksum y resultado de tests.
7. Separar almacenamiento de build de la fuente y restringir promoción.
8. Evaluar firma Authenticode antes de distribución pública.

Evidencia y criterio de gate

- Build record, checksum, log, test run y manifest Steam cuando exista.
- Comparación de contenido del ZIP/depot con lista permitida.

Criterio PASS de Hash manifest: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Build alterada o no atribuible.
- Falsos positivos y pérdida de confianza por binario sin reputación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

104. THIRD_PARTY_NOTICES

Objetivo y decisión

Este capítulo aplica desarrollo seguro al código, persistencia, builds y futura distribución sin atribuir al juego capacidades online que aún no existen. Para **THIRD_PARTY_NOTICES**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El registro legal mantiene revisión de LICENSE/NOTICE pendiente por paquete y bloquea distribución pública mientras no se cierre.

Controles y procedimiento

1. Validar entradas y estados antes de mutar dominio, archivos o economía.
2. Fallar de forma segura: no confirmar operaciones parciales ni ocultar corrupción.
3. Mantener separación entre Domain, Application, Infrastructure y Presentation.
4. Añadir tests de regresión específicos para cada control y condición excepcional.
5. Tratar licencia, procedencia y vulnerabilidad como dimensiones separadas.
6. Conservar texto legal exacto por versión y actualizar notices al cambiar paquetes.

Evidencia y criterio de gate

- Registro 23 y archivo de notices generado para la candidata.
- Hash y versión de cada dependencia distribuida.

Criterio PASS de THIRD_PARTY_NOTICES: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Distribuir software sin cumplir avisos.
- Confundir paquete oficial con licencia automáticamente resuelta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

105. Security test plan

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Security test plan**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Security test plan: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

106. Static analysis

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Static analysis**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Static analysis: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

107. Dependency scanning

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Dependency scanning**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada. La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.

4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Dependency scanning: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.
- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

108. Secret scanning automatizado

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Secret scanning automatizado**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.

7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Secret scanning automatizado: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

109. Malware scanning de artefactos

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Malware scanning de artefactos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada. El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.
9. Detener propagación y acceso sin borrar evidencia.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.
- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Malware scanning de artefactos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

110. Fuzzing selectivo

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Fuzzing selectivo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Fuzzing selectivo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

111. Tests de paths y archivos

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de paths y archivos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de paths y archivos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Falsa sensación de seguridad por cero findings.

- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

112. Tests de corrupción

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de corrupción**, la decisión por defecto es **PARTIAL / IMPLEMENTED AND TESTED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de corrupción: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

113. Tests de rollback

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de rollback**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: EditorBuildSettings.asset incluye Assets/_Project/Scenes/Bootstrap.unity, Assets/_Project/Scenes/MainMenu.unity, Assets/_Project/Scenes/Store.unity, Assets/_Project/Scenes/TestLab.unity. StoreInitial no está en el build profile y TestLab sí aparece, lo que bloquea una candidata pública. No existe CI/CD ni configuración SteamPipe demostrada. La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Generar build desde script reproducible y lista de escenas explícita.
6. Excluir TestLab, herramientas Editor, conceptos, fuentes y secretos.
7. Promover una misma candidata validada; no recompilar después del Go.
8. Conservar previous_stable y probar rollback antes de publicar.
9. Añadir pruebas por riesgo y no por herramienta.

Evidencia y criterio de gate

- Log de build, escenas incluidas, checksum y prueba de instalación.
- Manifest/branch y simulacro de rollback cuando Steam se abra.
- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de rollback: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Subir contenido interno o una escena de pruebas.
- No poder retirar rápidamente una actualización dañina.
- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 25_Post_Launch_and_Live_Operations_Plan.md.

114. Tests de permisos

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests de permisos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Tests de permisos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

115. Tests en usuario estándar

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Tests en usuario estándar**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera PLANNED hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Tests en usuario estándar: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

116. Clean machine test

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Clean machine test**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Clean machine test: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

117. Proton/Deck security review

Objetivo y decisión

Este capítulo convierte el control de seguridad en una prueba reproducible con entradas, expected result, evidencia y criterio de bloqueo. Para **Proton/Deck security review**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

HISTORICAL EVIDENCE — infraestructura automatizada retirada: La baseline funcional es 1215 EditMode + 70 PlayMode; no se ha demostrado SAST, SCA, secret scan remoto, malware scan de artefactos, fuzzing o campaña security-specific automatizada.

Controles y procedimiento

1. Definir fixture, versión, entorno, pasos, resultado esperado y evidencia.
2. Ejecutar sobre copia aislada; no utilizar credenciales o saves reales innecesariamente.
3. Tratar resultados no reproducibles como deuda, no como PASS implícito.
4. Bloquear la candidata cuando el test afecta integridad, secretos, permisos o rollback.
5. Añadir pruebas por riesgo y no por herramienta.
6. Conservar configuración y versión del scanner.
7. Triage humano de findings y prohibición de silenciar sin justificación.
8. Ejecutar clean-machine y usuario estándar sobre la build candidata.

Evidencia y criterio de gate

- Reporte del scanner y resolución por finding.
- Build, entorno, pasos y resultado de la campaña.

Criterio PASS de Proton/Deck security review: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Falsa sensación de seguridad por cero findings.
- Bloquear producción con ruido sin priorización de explotabilidad.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

118. Incident taxonomy

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Incident taxonomy**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, SECURITY.md, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Incident taxonomy: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

119. Severidades SEC-0 a SEC-4

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Severidades SEC-0 a SEC-4**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, SECURITY.md, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Severidades SEC-0 a SEC-4: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

120. Detección

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Detección**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, SECURITY.md, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Detección: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

121. Canales de reporte

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Canales de reporte**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, SECURITY.md, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Canales de reporte: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

122. Vulnerability disclosure

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Vulnerability disclosure**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, SECURITY.md, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Vulnerability disclosure: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

123. Recepción responsable de reportes

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Recepción responsable de reportes**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, SECURITY.md, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Recepción responsable de reportes: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

124. Triage de vulnerabilidad

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Triage de vulnerabilidad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe canal público de seguridad, SECURITY.md, bug bounty ni sistema de tickets. La taxonomía funcional S0-S4 no sustituye la severidad de seguridad SEC-0-SEC-4.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.

3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Definir evento, vulnerabilidad, incidente, crisis y brecha de datos.
6. Usar un canal privado; no pedir publicar una vulnerabilidad en foro o review.
7. Confirmar recepción sin prometer plazo no aprobado.
8. Reevaluar severidad tras obtener alcance y explotabilidad.

Evidencia y criterio de gate

- Incident/Vulnerability ID, timestamp y canal.
- Clasificación, impacto, activos y decisión.

Criterio PASS de Triage de vulnerabilidad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Divulgación pública prematura.
- Descartar un reporte por falta de exploit perfecto.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

125. Contención

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Contención**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Contención: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

126. Erradicación

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Erradicación**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Erradicación: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

127. Recuperación

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Recuperación**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.
9. Usar IDs estables, owner, target, evidencia y estado.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Recuperación: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.

- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

128. Post-incident review

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Post-incident review**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Post-incident review: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

129. Cadena de custodia

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Cadena de custodia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Cadena de custodia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

130. Preservación de evidencia

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Preservación de evidencia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Preservación de evidencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

131. Reloj y timeline

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Reloj y timeline**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Preservar copia antes de modificar el sistema afectado.
6. Revocar credenciales, aislar cuenta/build/host y conservar logs.
7. Eliminar causa raíz, rotar secretos y reconstruir desde fuente confiable.
8. Validar recuperación con tests y monitorización reforzada.

Evidencia y criterio de gate

- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.

Criterio PASS de Reloj y timeline: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

132. Comunicación interna

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Comunicación interna**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.

4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Comunicación interna: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 25_Post_Launch_and_Live_Operations_Plan.md.

133. Comunicación a jugadores

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Comunicación a jugadores**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Comunicación a jugadores: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 25_Post_Launch_and_Live_Operations_Plan.md.

134. Coordinación con Valve

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Coordinación con Valve**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Coordinación con Valve: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

135. Coordinación con proveedor

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Coordinación con proveedor**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Coordinación con proveedor: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

136. Brecha de datos

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Brecha de datos**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe soporte público ni Steamworks operativo. Las obligaciones concretas dependerán del incidente, contrato, jurisdicción y datos afectados.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Separar facts, hipótesis, impacto, acciones y próxima actualización.
6. Coordinar mensajes técnicos, legales, privacidad y comunidad.
7. Notificar al proveedor/Valve por el canal aplicable y conservar el case ID.
8. No minimizar ni exagerar antes de confirmar alcance.

Evidencia y criterio de gate

- Mensaje aprobado, fecha, audiencia y versión.
- Registro de contacto con plataforma/proveedor/autoridad cuando corresponda.

Criterio PASS de Brecha de datos: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Contradicciones públicas y pérdida de confianza.
- Retrasar indebidamente una notificación obligatoria.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 26_Privacy_Data_and_Telemetry_Plan.md.

137. Credencial comprometida

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Credencial comprometida**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Credencial comprometida: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

138. Repositorio comprometido

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Repositorio comprometido**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, loCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Repositorio comprometido: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

139. Paquete malicioso

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Paquete malicioso**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.

3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Paquete malicioso: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

140. Build alterada

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Build alterada**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Build alterada: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 24_Steam_Publishing_Plan.md.

141. Malware o ransomware

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Malware o ransomware**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Malware o ransomware: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

142. Pérdida de dispositivo

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Pérdida de dispositivo**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, loCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Pérdida de dispositivo: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

143. Publicación accidental

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Publicación accidental**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Publicación accidental: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

144. Secretos en commit

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Secretos en commit**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El scan textual no encontró patrones evidentes de secretos ni uso de variables de entorno. `.gitignore` excluye `.env`, `.env.*`, `*.keystore` y `*.jks`; Steamworks y SteamPipe no están integrados. El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Crear inventario de secretos por proveedor, scope, owner, fecha y mecanismo de revocación.
6. No guardar credenciales en VDF versionados, scripts, capturas, documentación, logs o chat.
7. Rotar inmediatamente ante exposición real o sospechada; borrar del repositorio no invalida el secreto.
8. Detener propagación y acceso sin borrar evidencia.
9. Revocar/rotar credenciales y revisar sesiones activas.

Evidencia y criterio de gate

- Secret scan del árbol y, cuando sea posible, del historial.
- Registro de emisión, rotación y revocación sin almacenar el valor secreto.
- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Secretos en commit: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Confundir `.gitignore` con protección del historial.
- Compartir credenciales para ahorrar tiempo y perder atribución.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: `09_CSharp_Coding_Standards.md`, `11_Build_y_Versioning_Guide.md`, `13_Trazabilidad_y_Control_de_Cambios.xlsx`.

145. Save exploit crítico

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Save exploit crítico**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Resolver y validar rutas dentro de un directorio permitido.
6. No confiar en nombres de archivo o JSON manipulados.
7. Escribir temporal, validar, rotar backup y reemplazar de forma controlada.
8. Limitar tamaños, recuentos y rangos antes de materializar el estado.
9. Detener propagación y acceso sin borrar evidencia.

Evidencia y criterio de gate

- Tests de truncado, checksum, schema, backup y restore.
- Casos de traversal, paths inválidos y archivos sobredimensionados.
- Scope confirmado, IoCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Save exploit crítico: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Borrado fuera del directorio previsto.
- Denegación de servicio local o economía inválida mediante save manipulado.
- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

146. Denegación de servicio futura

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Denegación de servicio futura**, la decisión por defecto es **NOT OPEN**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No se encontraron APIs de red, backend, autenticación propia, Steam ID ni servicios online. Todo este ámbito permanece NOT OPEN.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Abrir el gate solo con threat model, privacy review, ownership, authn/authz, rate limits, observabilidad y shutdown plan.
6. Usar TLS validado; prohibir certificados desactivados o secretos embebidos.
7. No usar Steam ID como secreto ni como autorización por sí solo.

Evidencia y criterio de gate

- ADR de apertura y revisión del proveedor.
- Tests de autenticación, autorización, abuso, rate limit y revocación.

Criterio PASS de Denegación de servicio futura: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Crear un backend sin capacidad de operación segura.
- Confiar en datos controlados por cliente o identificadores no verificados.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

147. Abuso de comunidad

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Abuso de comunidad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El escenario no se ha demostrado ocurrido; se documenta como runbook preventivo y debe adaptarse con datos reales durante el incidente.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.

3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Detener propagación y acceso sin borrar evidencia.
6. Revocar/rotar credenciales y revisar sesiones activas.
7. Identificar último estado confiable y reconstruir desde él.
8. Evaluar si jugadores, builds, datos, reputación o terceros están afectados.

Evidencia y criterio de gate

- Scope confirmado, loCs, commits/manifests afectados y acciones.
- Prueba de limpieza, restore y monitorización.

Criterio PASS de Abuso de comunidad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Cerrar al recuperar disponibilidad sin erradicar la causa.
- Publicar detalles explotables antes de desplegar mitigación.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 25_Post_Launch_and_Live_Operations_Plan.md.

148. Plan de crisis

Objetivo y decisión

Este capítulo define cómo reconocer, clasificar, contener, erradicar, recuperar y documentar un incidente sin destruir evidencia ni agravar el impacto. Para **Plan de crisis**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

El proyecto es de una sola persona y carece de equipo permanente de crisis; la simplicidad operativa es un requisito de seguridad.

Controles y procedimiento

1. Abrir un Incident ID, fijar severidad provisional y preservar el estado inicial.
2. Contener con el cambio mínimo que reduzca impacto sin destruir evidencia.
3. Separar erradicación de recuperación y validar que la causa raíz no persiste.
4. Cerrar con timeline, impacto, acciones, comunicación, evidencia y lecciones.
5. Mantener una hoja de una página con prioridades, contactos y decisiones irreversibles prohibidas.
6. Pausar releases y preservar cuentas/artefactos.
7. Solicitar apoyo profesional cuando el incidente supere capacidad técnica, legal o emocional.

Evidencia y criterio de gate

- Crisis log y decisiones Go/No-Go.
- Contactos verificados y simulacro.

Criterio PASS de Plan de crisis: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Tomar decisiones destructivas bajo presión.
- No comunicar por sobrecarga del único owner.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

149. Business continuity

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Business continuity**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Business continuity: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

150. RTO y RPO

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **RTO y RPO**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de RTO y RPO: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

151. Single-person dependency

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Single-person dependency**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

manifest.json declara 41 dependencias directas y packages-lock.json 57 entradas. Entre los paquetes directos figura com.unity.localization 1.5.12, además de Input System, URP, Test Framework y UGUI. El registro usa paquetes Unity oficiales/built-in y no muestra scoped registries personalizados; no se ha demostrado SBOM, Dependabot o vigilancia CVE automatizada. Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Versionar manifest y lockfile; revisar cambios directos y transitivos.
6. Actualizar una dependencia en rama, ejecutar tests y conservar rollback.
7. Eliminar paquetes sin uso y evitar fuentes no oficiales.
8. Registrar vulnerabilidad, explotabilidad real, workaround y decisión.
9. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.

Evidencia y criterio de gate

- Diff de manifest/lock, inventario SBOM y fuente del paquete.
- Test run y build de compatibilidad tras la actualización.
- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Single-person dependency: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Supply-chain compromise o paquete abandonado.
- Actualizar automáticamente y romper el proyecto o introducir comportamiento no revisado.
- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx, 23_Legal_Credits_and_Licenses_Register.xlsx.

152. Recuperación de cuentas

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Recuperación de cuentas**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario. Antes de este documento no existía un runbook de incidente integrado; sí existen patrones técnicos de backup-first recovery para saves y prácticas de build/rollback históricas. La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.
9. Preservar copia antes de modificar el sistema afectado.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.
- Timeline UTC, hashes, capturas, logs y lista de acciones.
- Postmortem sin culpabilización y acciones con owner.
- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Recuperación de cuentas: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.
- Destruir evidencia durante la limpieza.
- Volver a producción con persistencia del atacante o causa raíz abierta.
- Convertir la documentación en cumplimiento de papel.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

153. Contactos de emergencia

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Contactos de emergencia**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

Existen paquetes ZIP y documentación archivada, pero no se ha demostrado una política 3-2-1, copia offline/inmutable, restauración periódica ni sustituto operativo para el único propietario.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Mantener al menos tres copias, dos medios y una fuera del fallo común cuando sea viable.
6. Definir RPO/RTO por repositorio, arte fuente, documentos, cuentas y builds.
7. Probar restauración selectiva y pérdida total.
8. Conservar instrucciones de recuperación sin incluir secretos.

Evidencia y criterio de gate

- Fecha de última copia y último restore test.
- Inventario de cuentas, recovery codes y contactos custodiado de forma segura.

Criterio PASS de Contactos de emergencia: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Backups corruptos o sincronizados con el mismo ransomware.
- Imposibilidad de continuar por dependencia de una sola persona.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

154. Simulacros

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Simulacros**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

No existe todavía evidencia específica suficiente; el control se considera PLANNED hasta que se ejecute y archive una prueba.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.

Evidencia y criterio de gate

- Registro fechado del control y su resultado.
- Referencia a commit, build, configuración o incidente aplicable.
- Owner y riesgo residual.

Criterio PASS de Simulacros: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Control declarado pero no ejecutado.
- Evidencia insuficiente para una decisión de release.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

155. Métricas de seguridad

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Métricas de seguridad**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Usar IDs estables, owner, target, evidencia y estado.
6. Cerrar deuda solo con prueba; una aceptación necesita vencimiento y riesgo residual.
7. Revisar gates al cambiar alcance, proveedor o build.

Evidencia y criterio de gate

- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Métricas de seguridad: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Convertir la documentación en cumplimiento de papel.
- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

156. Security debt

Objetivo y decisión

Este capítulo mantiene el plan ejecutable mediante continuidad, métricas, deuda, gates, checklists, plantillas y trazabilidad. Para **Security debt**, la decisión por defecto es **PLANNED**: ninguna afirmación de seguridad se considera cerrada sin evidencia repetible, y ningún sistema futuro queda autorizado por el mero hecho de estar descrito.

Estado actual comprobable

La capacidad está principalmente planificada; las métricas deben mostrar reducción de riesgo y readiness, no solo volumen de herramientas.

Controles y procedimiento

1. Mantener el procedimiento breve para crisis y la documentación extensa para preparación.
2. Asignar owner, fecha objetivo, dependencia, evidencia de cierre y riesgo residual.
3. Revisar el control tras incidentes, cambios de proveedor o cambios de arquitectura.
4. Actualizar Binder, Guía Maestra, Producción, Trazabilidad y checklist H6 cuando aplique.
5. Usar IDs estables, owner, target, evidencia y estado.
6. Cerrar deuda solo con prueba; una aceptación necesita vencimiento y riesgo residual.
7. Revisar gates al cambiar alcance, proveedor o build.

Evidencia y criterio de gate

- Registro actualizado y enlaces a pruebas.
- Historial de cambios del plan.

Criterio PASS de Security debt: la evidencia debe corresponder a la versión, commit, cuenta, workstation o build evaluada; una baseline histórica no sustituye la revalidación del objeto actual.

Riesgos y tratamiento

- Convertir la documentación en cumplimiento de papel.
- Mantener controles obsoletos sin ejecutar.

Toda desviación de este capítulo requiere owner, exposición, control compensatorio, vencimiento y decisión explícita. No se admite aceptación tácita cuando pueda afectar secretos, integridad de build, pérdida de save, publicación o datos personales.

Trazabilidad

Fuentes operativas principales: 09_CSharp_Coding_Standards.md, 11_Build_y_Versioning_Guide.md, 13_Trazabilidad_y_Control_de_Cambios.xlsx.

157. Work packages

WP	Alcance	Objetivo	Evidencia de cierre
SEC-WP-01	Inventario de activos, cuentas y trust boundaries	S17	Asset/account register + own
SEC-WP-02	MFA, password manager y recovery	S17	Captura/registro sin secretos
SEC-WP-03	Backup 3-2-1 y restore drill	S17/H6	Restore report
SEC-WP-04	SECURITY.md y canal privado	Pre-public	Published policy
SEC-WP-05	Secret scan de árbol e historial	S17	Scan report + remediation
SEC-WP-06	Branch protection y revisión compensatoria	S17	Repository settings
SEC-WP-07	SBOM y dependency advisory process	S17/H6	SBOM + triage log
SEC-WP-08	Auditoría UnityConnect/ProjectSettings	S17	Settings decision + network o
SEC-WP-09	Tests de paths, borrado y save tampering	S17	EditMode/PlayMode evidence
SEC-WP-10	Build allowlist y exclusión TestLab	H6	Build contents report
SEC-WP-11	Signing/reputation decision Windows	Pre-Steam	ADR and certificate plan
SEC-WP-12	Steamworks IAM y SteamPipe secrets	Pre-Steam	Roles, MFA, upload runbook
SEC-WP-13	Incident register y templates	S17	Dry run
SEC-WP-14	Tabletop: leaked secret	S17	Exercise report
SEC-WP-15	Tabletop: bad build/rollback	H6	Exercise report
SEC-WP-16	Malware scan y clean-machine campaign	H6	Scanner + clean install eviden
SEC-WP-17	Vulnerability monitoring and patch SLA	Ongoing	Advisory log
SEC-WP-18	External security review trigger	Pre-release	Scope/decision
SEC-WP-19	Emergency continuity packet	H6	Custodied recovery instruction
SEC-WP-20	Final security Go/No-Go	H6/Steam	Signed decision

Cada WP tiene owner, dependencias, riesgo, coste estimado y criterio de rollback. Un WP no se cierra con “configurado”: debe conservar resultado reproducible y reabrirse cuando cambie el activo o proveedor.

158. Gates

Gate de desarrollo seguro

- árbol e historial sin secretos activos;
- dependencias inventariadas y lockfile revisado;
- tests afectados en PASS;
- ProjectSettings/Packages diffs aprobados;
- operaciones destructivas limitadas y probadas.

Gate H6

- cero SEC-0/SEC-1 abiertos;
- restore drill de repositorio y save;
- build limpia sin TestLab, tools, secrets o assets NOT OPEN;
- secret/dependency/malware scans revisados;
- rollback probado;
- incident templates y contactos listos;
- riesgo residual firmado.

Gate Steamworks

- cuenta individual, MFA, roles mínimos y recovery;
- VDF/secrets fuera de Git;
- upload desde workstation confiable;
- manifests/branches trazados;
- previous stable y revocación disponibles.

Gate de sistema online

Backend, Cloud, telemetría, Workshop o cuentas requieren threat model, privacidad, authn/authz, abuse controls, observabilidad, incident response, proveedor y shutdown plan. Hasta entonces: NOT OPEN.

159. Checklist pre-commit

- ☐ Diff limitado y entendible
- ☐ No secretos, tokens, VDF sensibles ni datos personales
- ☐ No binarios inesperados
- ☐ Tests targeted ejecutados
- ☐ Manifest/lock revisado si cambia
- ☐ Paths y borrados revisados
- ☐ Commit message y issue/ADR enlazados
- ☐ Working tree limpio tras commit

La checklist se adjunta al commit/build/release correspondiente. Los elementos no aplicables deben indicar motivo y aprobación; no se dejan en blanco.

160. Checklist pre-build

- ☐ Commit y versión identificados
- ☐ Secret scan
- ☐ Dependency/SBOM review
- ☐ Suites y targeted tests
- ☐ Lista de escenas aprobada
- ☐ TestLab y tools excluidos
- ☐ Development Build/script debugging según objetivo

- ☐ Artefactos previos eliminados o aislados
- ☐ Build en ruta controlada
- ☐ Checksum y log archivados
- ☐ Malware scan
- ☐ Ejecución como usuario estándar

La checklist se adjunta al commit/build/release correspondiente. Los elementos no aplicables deben indicar motivo y aprobación; no se dejan en blanco.

161. Checklist pre-release

- ☐ Candidata congelada
- ☐ Build/depot coincide con artefacto probado
- ☐ StoreInitial integrada
- ☐ Golden Path y siete días
- ☐ Install/update/uninstall
- ☐ Save migration/recovery
- ☐ Previous stable y rollback
- ☐ Créditos/notices/licencias
- ☐ Localización y privacidad
- ☐ MFA/cuentas/roles
- ☐ SEC-0/SEC-1 = 0
- ☐ Known issues y soporte
- ☐ Go/No-Go firmado

La checklist se adjunta al commit/build/release correspondiente. Los elementos no aplicables deben indicar motivo y aprobación; no se dejan en blanco.

162. Plantilla de incidente

Incident ID:
Título:
Fecha/hora UTC de detección:
Reportado por / canal:
Incident Commander:
Severidad provisional / final:
Activos y versiones afectados:
Indicadores y evidencia preservada:
Impacto confirmado / potencial:
Datos personales implicados:
Contención aplicada:
Credenciales revocadas/rotadas:
Causa raíz:
Erradicación:
Recuperación y tests:
Comunicación interna/externa:
Proveedor/Valve/autoridad contactados:
Timeline UTC:
Riesgo residual:
Acciones y owners:
Fecha de cierre:
Postmortem:

163. Plantilla de vulnerability report

Vulnerability ID:
Versión/build/commit:
Componente:
Descripción técnica:
Precondiciones:

Pasos de reproducción o PoC seguro:

Impacto:

Alcance:

Datos o secretos expuestos:

Mitigación temporal:

Divulgación previa/pública:

Contacto del reportante:

Permiso para acreditar:

Adjuntos y hashes:

No se exige al reportante ejecutar acciones destructivas, acceder a datos de terceros ni publicar la vulnerabilidad. La respuesta inicial confirma recepción y proporciona un identificador; no promete recompensa o fecha si no existe programa aprobado.

164. Plantilla de risk acceptance

Risk ID:

Activo/sistema:

Amenaza y escenario:

Probabilidad / impacto / exposición:

Control faltante:

Motivo para no corregir ahora:

Compensating controls:

Versiones/builds autorizadas:

Owner:

Fecha de expiración:

Condiciones de reapertura:

Aprobador:

Evidencia:

No se acepta de forma indefinida un riesgo de secreto activo, cuenta de publicación comprometida, build manipulada, pérdida generalizada de save, malware, incumplimiento legal o exposición de datos personales.

165. RACI

Actividad	Owner/IC	Desarrollo	QA	Legal/Privacidad	Plataforma/Proveedor	Revisor
Inventario y controles	A/R	R	C	C	I	C
Patch de dependencia	A	R	R	C	C	C
SEC-0/SEC-1	A/R	R	R	C	C	C
Brecha de datos	A	C	C	R	C	C
Steam credential incident	A/R	C	C	C	R/C	C
Release Go/No-Go	A	R	R	C	C	C
Comunicación pública	A	C	C	C	C	C

En el estado actual, el owner acumula funciones. A/R no elimina la necesidad de revisión compensatoria para una publicación pública o incidente severo.

166. Definition of Ready

Un trabajo con impacto de seguridad está Ready cuando:

- activo y owner están identificados;
- amenaza, trust boundary y datos están descritos;
- dependencias y proveedor son conocidos;
- existe plan de test, rollback y evidencia;
- secretos/roles se han diseñado sin valores embebidos;

- legal, privacidad y licencias han sido consultados cuando aplica;
- no abre silenciosamente un sistema NOT OPEN;
- el riesgo de no hacerlo también está registrado.

167. Definition of Done

Un control o cambio está Done cuando:

- implementación y configuración están versionadas;
- revisión y tests pasan en el entorno objetivo;
- el resultado negativo también se ha probado cuando es relevante;
- no existen secretos o datos inesperados en código, logs y build;
- documentación, threat model, SBOM y notices están actualizados;
- recuperación/rollback se ha ejecutado, no solo descrito;
- findings se han cerrado o aceptado con vencimiento;
- la evidencia está archivada y enlazada;
- el gate afectado se ha actualizado manualmente.

168. Riesgos abiertos

Risk ID	Riesgo	Nivel inicial	Tratamiento mínimo
SEC-RISK-001	Single-person dependency	High	Backup, emergen
SEC-RISK-002	No restore drill	Critical	3-2-1 backup and
SEC-RISK-003	No remote secret/history scan evidence	High	Scan history and
SEC-RISK-004	No dependency advisory process	High	SBOM and period
SEC-RISK-005	TestLab included in build profile	High	Release allowlist
SEC-RISK-006	StoreInitial absent from build profile	High	Controlled migrat
SEC-RISK-007	Unity diagnostics settings ambiguous	Medium	Audit settings an
SEC-RISK-008	No code-signing decision	Medium	Authenticode/rep
SEC-RISK-009	No SECURITY.md/private channel	High	Publish before ex
SEC-RISK-010	No Steamworks IAM yet designed operationally	High	MFA/roles/recover
SEC-RISK-011	File deletion paths insufficiently security-tested	Medium	Path boundary an
SEC-RISK-012	Save checksum may be mistaken for anti-tamper	Medium	Document limits
SEC-RISK-013	No malware scan/clean machine evidence	High	Candidate scan a
SEC-RISK-014	No tabletop exercises	Medium	Secret leak and b
SEC-RISK-015	No external review trigger defined	Medium	Trigger by online
SEC-RISK-016	Package LICENSE/NOTICE still open	High	Coordinate with l
SEC-RISK-017	No secure support channel	Medium	Create with priva
SEC-RISK-018	Build reproducibility not proven across clean environment	High	Clean workstation
SEC-RISK-019	Backups may share same failure domain	High	Offline/offsite cop
SEC-RISK-020	Future AI/tool uploads may expose confidential data	Medium	Tool data policy a

Los niveles son de planificación, no una evaluación legal definitiva. Mientras no se genere el futuro 33_Project_Risk_Register_and_Business_Continuity_Plan.xlsx, este capítulo y 13_Trazabilidad_y_Control_de_Cambios.xlsx conservan el contexto y seguimiento operativo de seguridad.

169. Glosario

Término	Definición operativa
Activo	Elemento con valor para el proyecto.
Advisory	Aviso sobre vulnerabilidad o riesgo de un componente.
Build provenance	Evidencia de origen y proceso de una build.
Causa raíz	Condición que permitió el incidente y cuya corrección previene recurrencia.
Contención	Acción para limitar impacto inmediato.
CVE	Identificador público de una vulnerabilidad conocida.
Evento	Observación que puede o no ser incidente.
Incidente	Evento que compromete o amenaza confidencialidad, integridad, disponibilidad.
IoC	Indicador de compromiso.
MFA	Autenticación con más de un factor.
RPO	Pérdida máxima de datos tolerada medida en tiempo.
RTO	Tiempo objetivo para recuperar un servicio/activo.
SBOM	Inventario de componentes de software y versiones.
Secret	Credencial que concede acceso o firma.
SAST	Análisis estático de seguridad.
SCA	Análisis de componentes y dependencias.
Threat model	Modelo de activos, actores, límites y amenazas.
Trust boundary	Punto donde cambia el nivel de confianza.
Vulnerability disclosure	Proceso coordinado de recepción, corrección y comunicación.
Zero-day	Vulnerabilidad sin corrección disponible o desconocida previamente.

170. Fuentes oficiales

Fuentes verificadas el **2026-07-01**. Deben revalidarse antes de una decisión externa, especialmente si cambian GitHub, Steamworks, Unity, NIST u OWASP.

ID	Fuente	URL
NIST-CSF-2.0	NIST Cybersecurity Framework 2.0	https://www.nist.gov/cybersecurity-framework
NIST-SSDF-1.1	NIST SP 800-218 — Secure Software Development Framework 1.1	https://csrf.nist.gov
NIST-IR-3	NIST SP 800-61 Rev. 3 — Incident Response Recommendations	https://csrf.nist.gov
CISA-SBD	CISA Secure by Design	https://www.cisa.gov/secure-by-design
OWASP-A03-2025	OWASP Top 10:2025 — A03 Software Supply Chain Failures	https://owasp.org/Top10/A03-2025-Software-Supply-Chain-Failures
GITHUB-SECRETS	GitHub Docs — Secret scanning	https://docs.github.com/en/code-security/secret-scanning
GITHUB-BRANCHES	GitHub Docs — About protected branches	https://docs.github.com/en/code-security/secret-scanning
GITHUB-SECURITY-POLICY	GitHub Docs — Adding a security policy	https://docs.github.com/en/code-security/secret-scanning
GITHUB-DEPENDABOT	GitHub Docs — Dependabot alerts	https://docs.github.com/en/code-security/secret-scanning
STEAM-USERS	Steamworks — Managing Your Steamworks Account	https://partner.steamgames.com/doc/steamworks/creating-account
STEAM-UPLOAD	Steamworks — Uploading to Steam	https://partner.steamgames.com/doc/steamworks/creating-account
UNITY-UPM	Unity Manual — Switch to another version of a UPM package	https://docs.unity3d.com/Manual/switching-upm-versions.html

Las referencias se usan como guía proporcional. No se afirma cumplimiento certificado ni se copian requisitos de entornos empresariales que no aportan valor al proyecto.

171. Inventario histórico

Se han conservado **234 fuentes históricas u operativas relevantes**. El filtro es deliberadamente amplio e incluye builds, QA, ADR, handoffs, legal, paquetes, Steam, persistencia y auditorías; una fuente listada puede ser contextual y no una autoridad de seguridad autónoma.

|---|---|---|---:|---|

Documentos consolidados actuales consultados

172. Historial de cambios

Fecha	Versión	Cambio
2026-07-01	v1.0-consolidated	Creación del plan autónomo a partir de baselines v0.3-v0.6, ADR, código
2026-07-01	v1.1-verified	Incorporación de NIST SP 800-61 Rev. 3, estado técnico real, package i

Próxima revisión obligatoria

- antes de onboarding Steamworks;
- antes de external playtest o beta;
- antes de H6/release candidate;
- al añadir backend, Cloud, telemetría, crash reporting, Workshop o colaboradores;
- tras cualquier SEC-0, SEC-1 o near miss relevante.

Este historial registra cambios del documento. La ejecución de controles y el cierre de riesgos se registran en Producción, Trazabilidad, H6 y Risk Register.